

基于Syslab的科学计算应用 培训签到



请各位老师扫码签到

MoHub技术交流专区/高校专区 MWORKS技术答疑和交流社区



直接搜索 “MoHub”

<https://mohub.net>

基于MWORKS.Syslab的 科学计算应用

孔德才

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

传播或以其他方式使用

目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

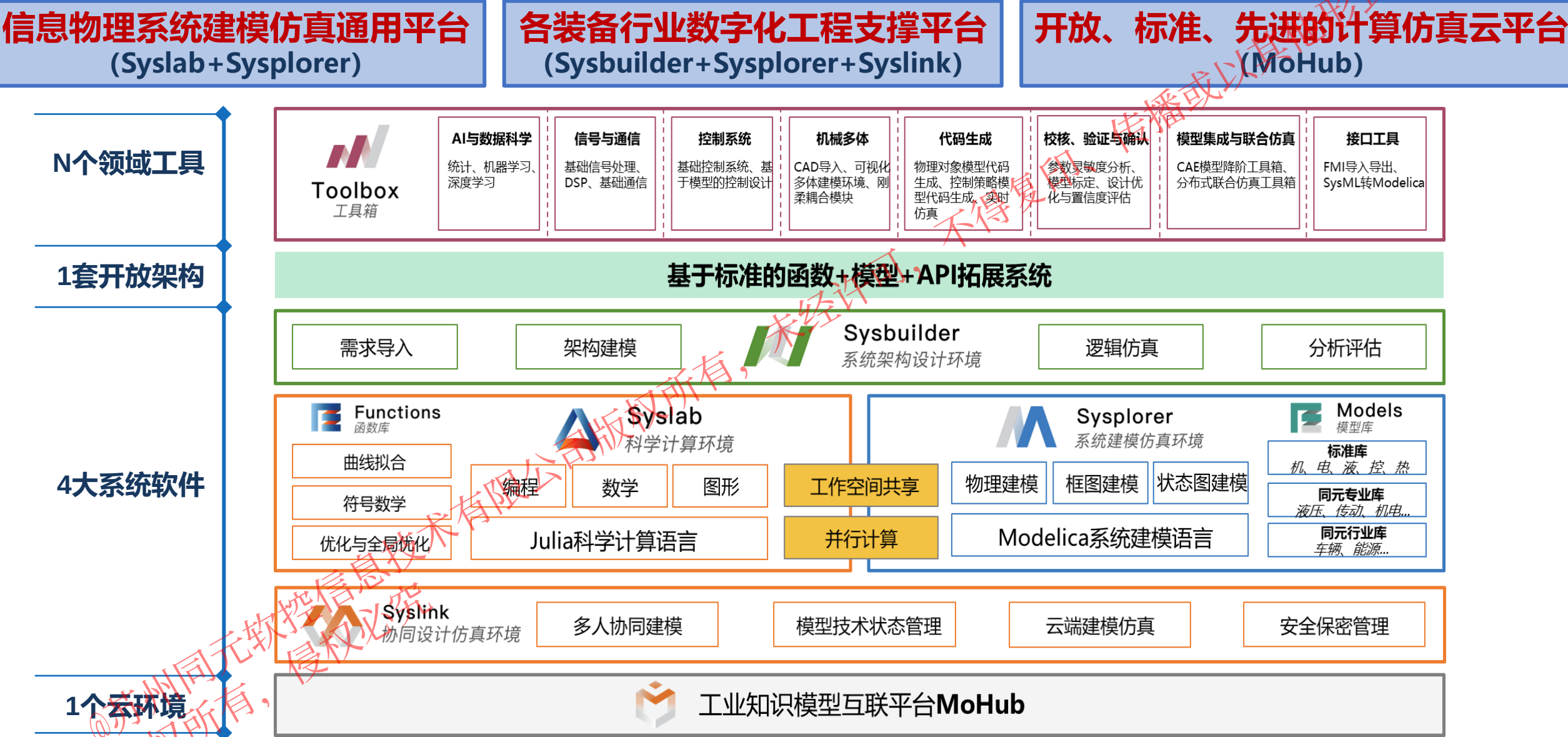
3. 工具箱使用

4. Julia脚本构建与调试

5. 结果后处理

6. 与其他语言的差异

1.1 新一代科学计算与系统建模仿真平台MWORKS简介-框架



1.1 新一代科学计算与系统建模仿真平台MWORKS简介-框架

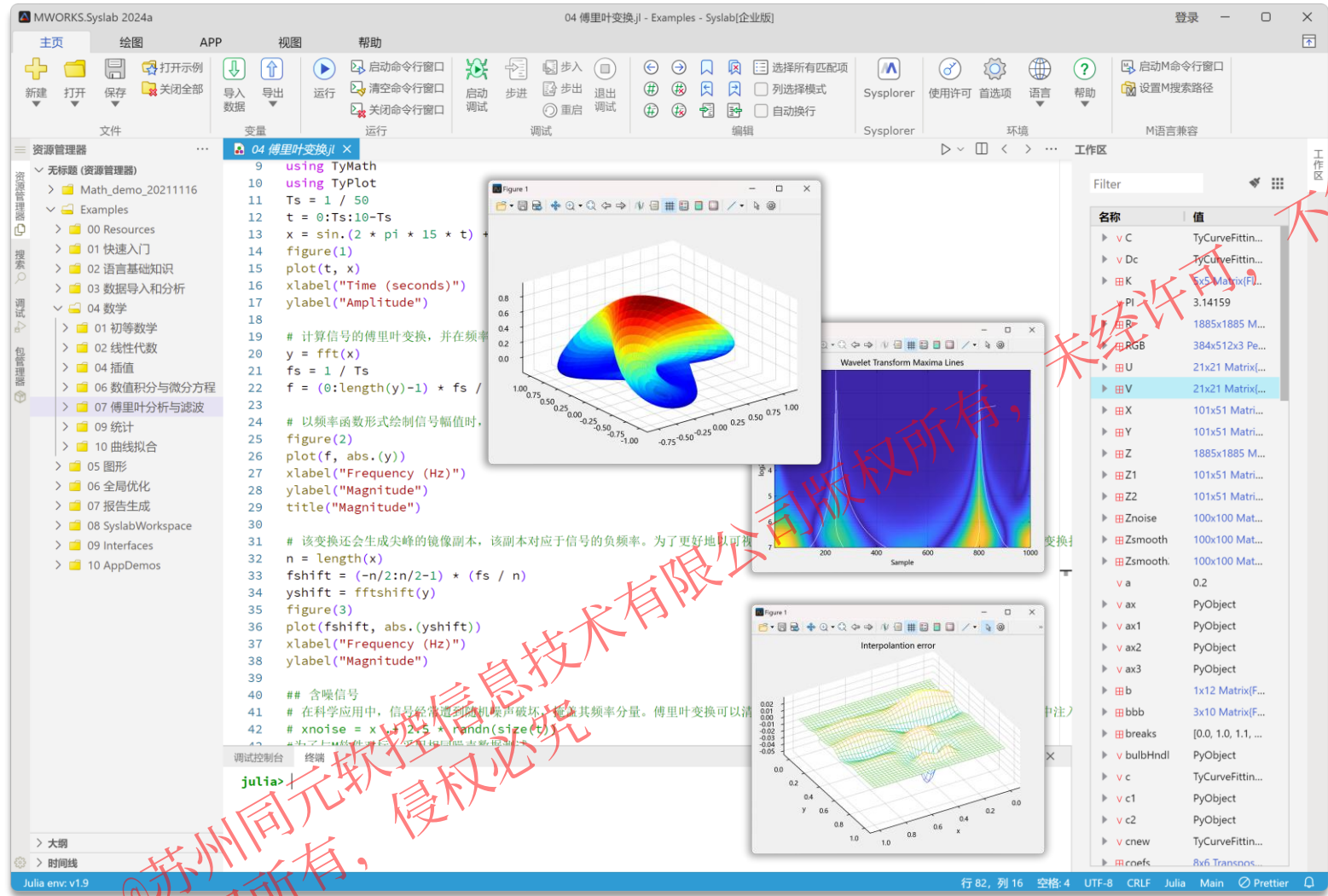
MWORKS正在成为**中国的MATLAB**，是全球**第四**个科学计算与建模仿真一体化软件

MWORKS经过 **20 余年**的持续研发和技术积累，在功能层面达到与MATLAB/Simulink相当水平



1.2 科学计算环境 Syslab

Syslab 用于算法开发与数值计算、数据分析与可视化、信息域计算分析等场景



数学计算引擎

提供基础数学、符号数学、统计、优化等大量高质量内置函数，支撑科学与工程计算

通用编程与算法开发环境

基于高级通用动态编程语言支撑高性能算法开发

数据分析与可视化

对数据进行探查、分析与可视化

领域工具箱基础支撑环境

通过平台基础功能支撑领域工具箱的开发与运行



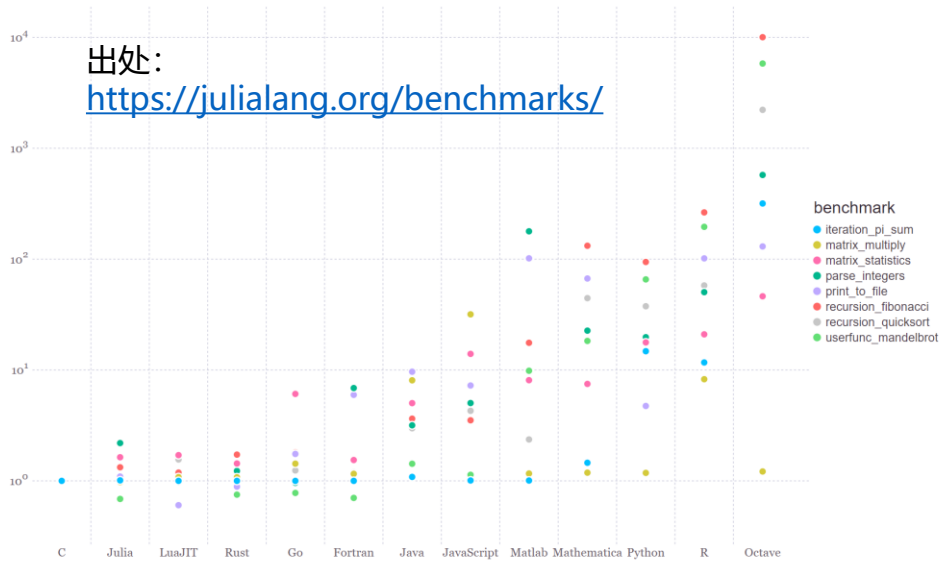
1.2 科学计算环境 Syslab

MWORKS.Syslab算法语言：Julia语言

Julia 语言是一门高级、通用、动态的程序设计语言，适合用于与科学计算相关的高性能计算领域。



功能	Julia	M语言	Python
创建1维数组	A=[1;2;3]或 A=[1,2,3]	无法创建	A=np.array[1,2,3]
创建行、列矩阵	行: A=[1 2 3] 列: A=[1 2 3]'	行: A=[1 2 3] 列: A=[1;2;3]	行: A=np.array[1 2 3].reshape(1,3) 列: A=np.array[1 2 3].reshape(3,1)
K个点的线性间隔向量	A=range (1,5,length=k)	A=linspace(1,5,k)	A=np.linspace(1,5,k)
数组、字符串等的索引	从1开始; 不支持负数索引; 索引必须写全, 例如a[2:end]	从1开始; 不支持负数索引; 索引必须写全, 例如a[2:end]	从0开始; 支持负数索引; 索引可不写全, 例如a[1:]
矩阵乘法	A*B代表矩阵相乘, A.*B代表逐元素相乘	A*B代表矩阵相乘, A.*B代表逐元素相乘	A*B代表逐元素相乘, A@B代表矩阵相乘
函数调用	y1,y2=f(x1,x2) 默认参数重新求值	[y1,y2]=f(x1,x2) /	[y1,y2]=f([x1,x2]) 只在函数定义是对默认参数求一次值
...	...	...	...



Julia语言与M语言和Python语言语法较为类似，官网语法课程详细介绍语法。

在遵循Julia编码规范下，可获得更高的计算效率。



1.3 科学计算环境Syslab-目标群体



面向**算法开发**的工程师:

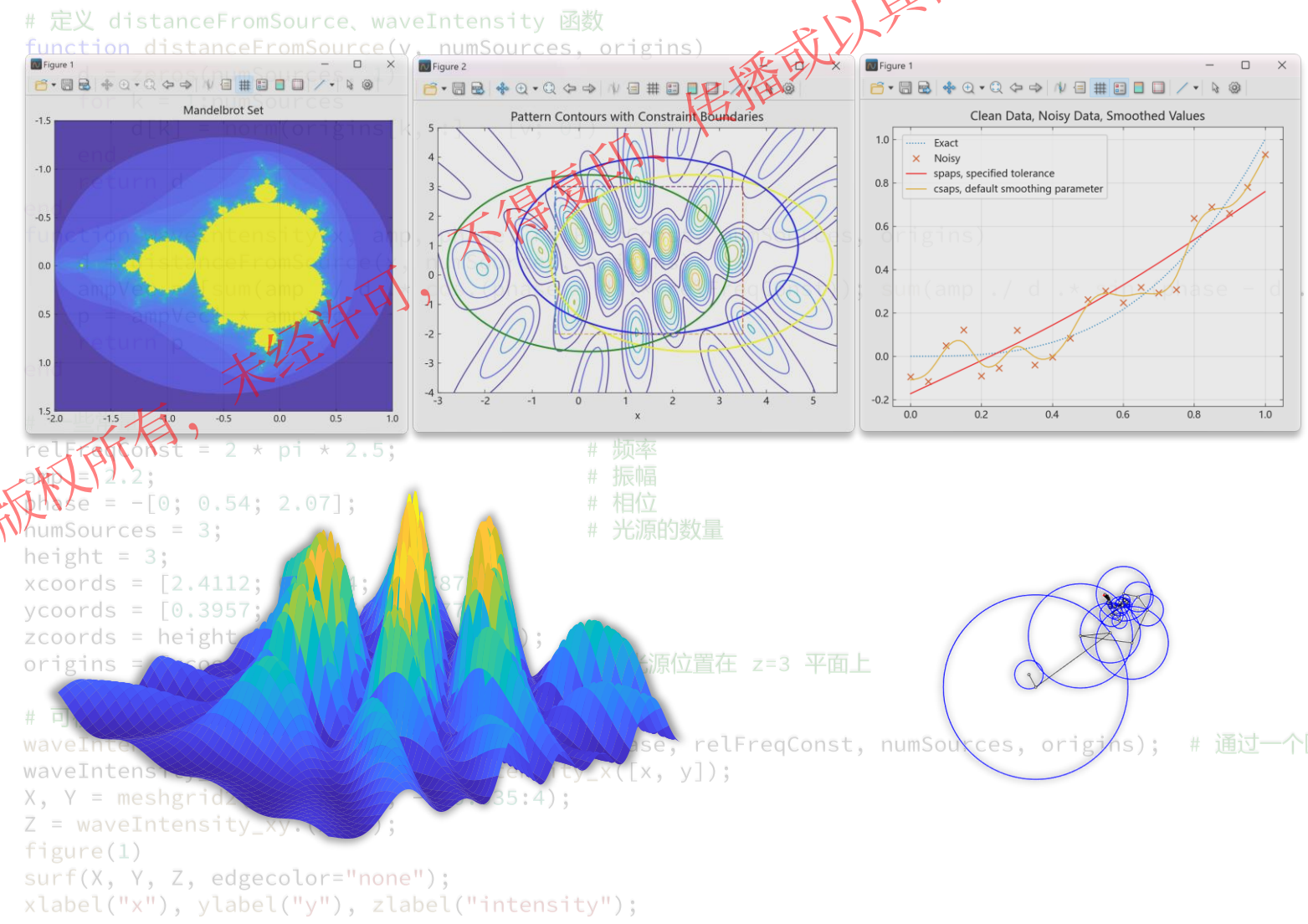
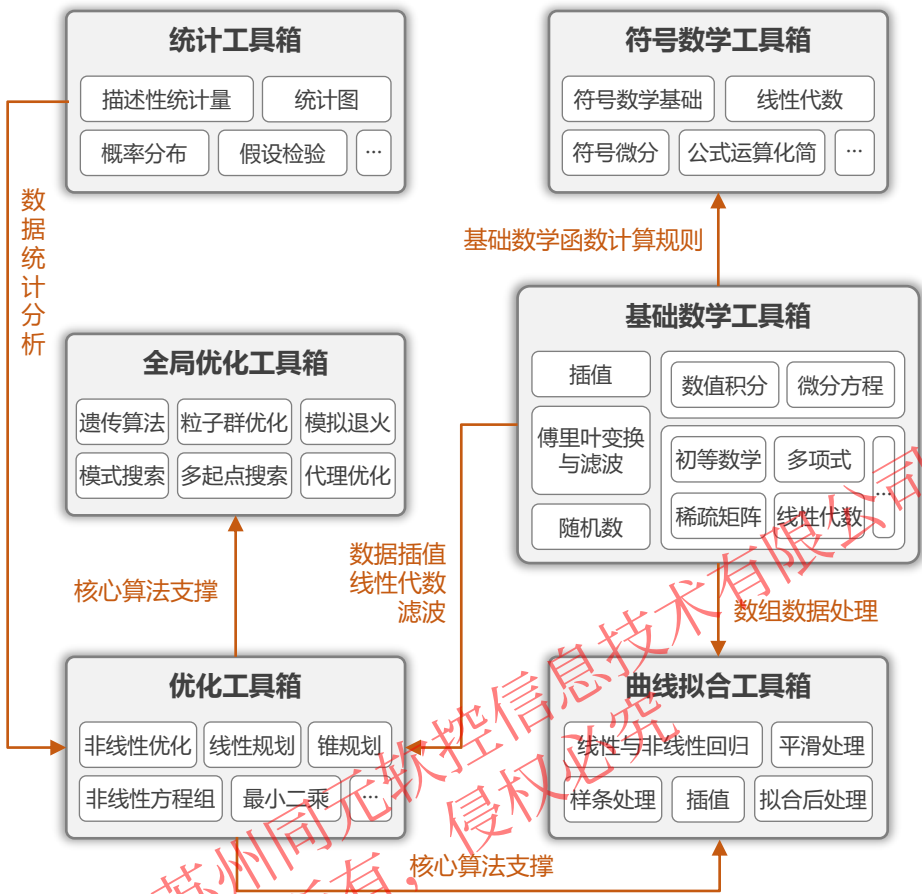
- 信号处理算法开发
- 控制系统算法开发
- 机器学习算法开发
- 深度学习算法开发
- 大数据处理算法开发
- 图像处理算法开发

.....
@苏州同元软控信息技术有限公司
版权所有，侵权必究



1.4 科学计算环境 Syslab - 为科学与工程问题提供丰富高性能的数学库

通过**基础数学**、**符号数学**、**曲线拟合**、**统计**、**优化**、**全局优化**6个工具箱，超过1500个函数支撑科学与工程计算，同时为上层专业工具箱提供计算支撑



1.4 科学计算环境 Syslab - 工程师的高级编程语言

交互式编程，不需要编程经验

```
julia> log10(2.5)
0.3979400086720376
```

以数学表达形式直接表示矩阵或数组

```
julia> A = [8 1 6; 3 5 7; 4 9 2]
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2
```

Syslab为基础数学、科学和工程计算提供了大量的内置函数

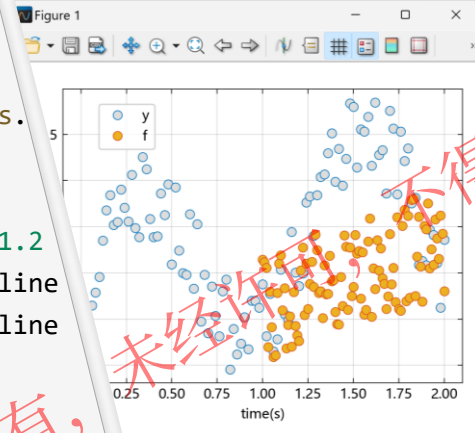
```
julia> eigvals(A)
3-element Vector{Float64}:
-4.8989794855663575
 4.898979485566359
15.000000000000004

julia> inv(A)
3×3 Matrix{Float64}:
 0.147222  -0.144444  0.0638889
-0.0611111  0.0222222  0.105556
-0.0194444  0.188889  -0.102778
```

将命令组合为脚本

组合命令创建脚本，添加高级编程构造，如条件语句和循环

```
x = 0:0.02:2
α = 1:0.01:2
y = 2 * sin.(3 * x) .* cos.
f = log.(α)
noise1 = rand(length(x))
noise2 = rand(length(α))*1.2
plot(x, y + noise1, "o", line
plot(α, f + noise2, "o", line
grid("on")
legend("y", "f")
```



编写可重用函数

使用函数将复杂程序分解为更小的可重用部分

```
function fibonacci(n::Integer)
    n <= 1 ? n : fibonacci(n - 1) + fibonacci(n - 2)
end

julia> fibonacci(1)
1

julia> fibonacci(10)
55
```

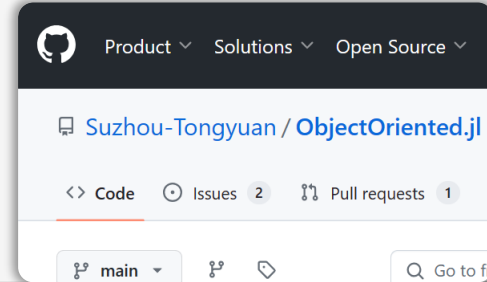
面向对象的编程

支持创建自定义类，进而实现将代码组织成组件使其更易于维护和扩展。面向对象的编程通过ObjectOriented.jl库实现，该库是同元软控研发，**现已被Julia社区集成。**

```
@oostruct MutableData
x::Int
y::Int

function new(x::Int, y::Int)
    @mk begin
        x = x
        y = y
    end
end

mt = MutableData(1, 2)
mt.x += 1
```



大型算法程序/工具箱开发

使用 Syslab和Julia语言开发和维护大型应用程序，支撑工程、源代码管理集成、单元测试、持续集成和部署以及打包等

1.4 科学计算环境 Syslab - 数据分析、探查与可视化

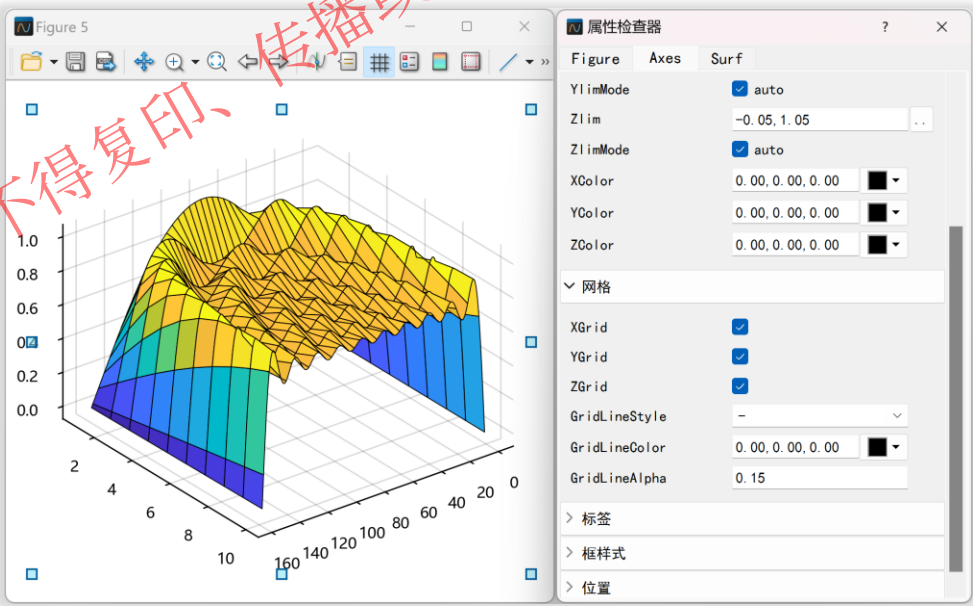
通过内置图形库创建数据可视化

超过220个绘图函数，涵盖线图、数据分布图、极坐标图、等高线图、向量场、曲面、图像、地理图等

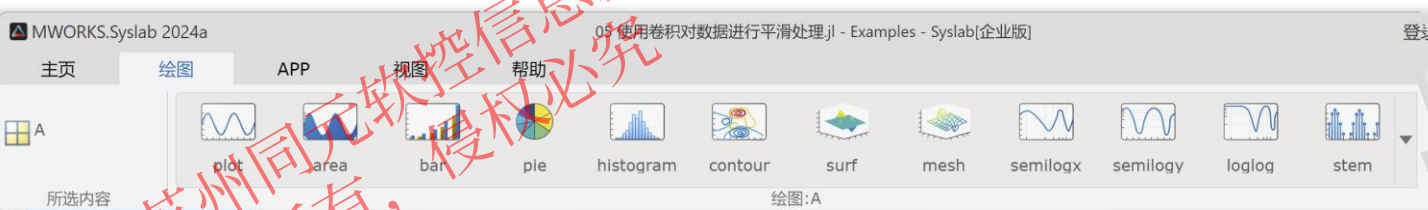


使用交互式工具自定义可视化

易用的图窗交互功能，支持自定义绘图外观



通过Syslab绘图工具创建可视化图形



高质量图形输出

支持将绘图保存为syslabfig文件，同时支持直接导出出版级质量的图形，用于论文、海报和演示，或者保存为图像或矢量图文件，支持的格式包括：.jpg、.png、.raw、.svg、.tiff等

1.4 科学计算环境 Syslab - 详细的帮助手册和丰富的示例

为每个函数提供了详尽语法、使用说明，并提供了可运行的代码示例片段

丰富的综合示例，可在 Syslab 中一键打开和运行，方便用户快速学习

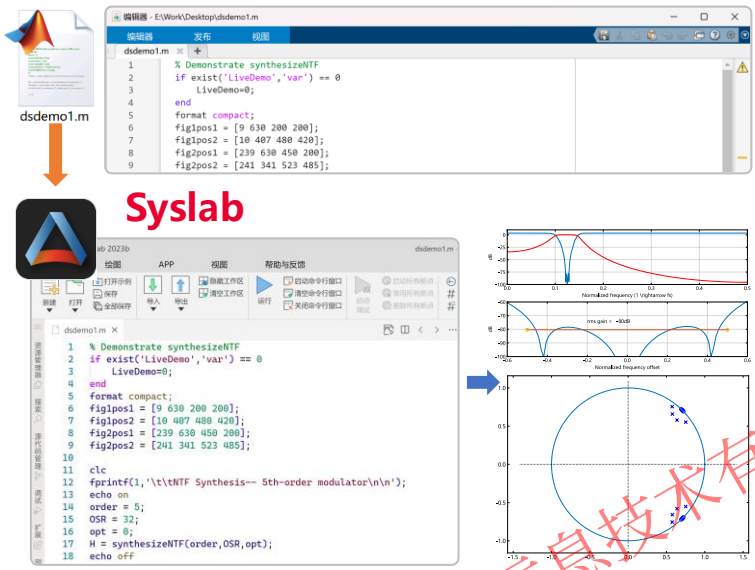


1.4 科学计算环境 Syslab – M语言兼容

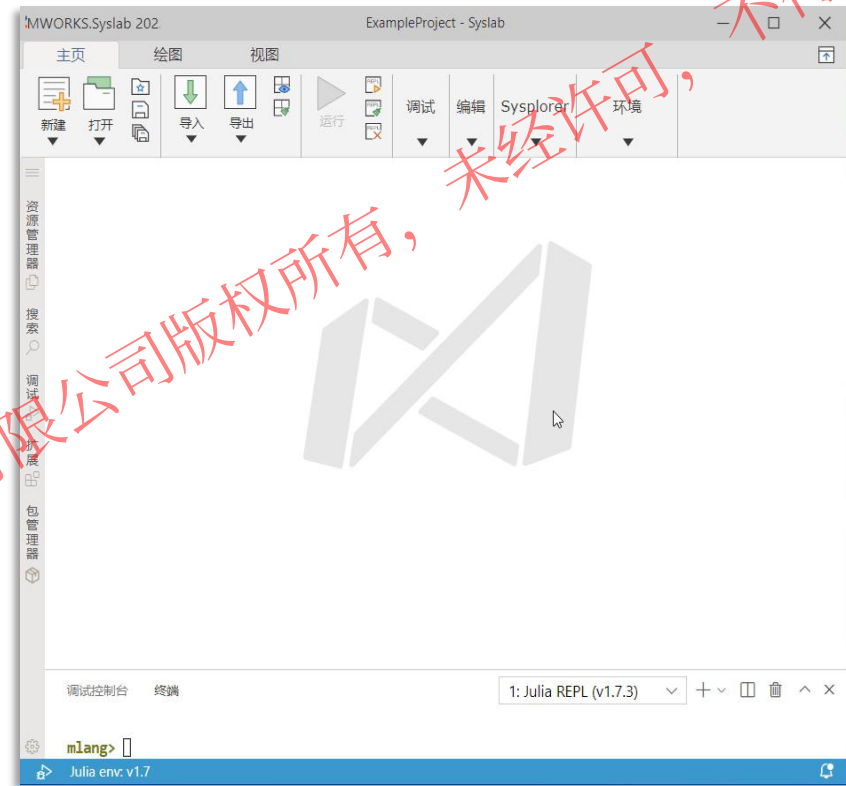
- ❑ **打造M兼容工具：**兼容M生态实现用户历史资产重用
- ❑ **M语言无缝接入Julia科学计算生态：**M语言语法兼容度超97%，内置1560+个常用函数，涵盖基础、数学、图形、控制、信号处理等领域常用函数，无须安装MATLAB直接运行M代码
- ❑ 提供M函数映射工具，能够调用Julia先进的科学计算库，**支持用户自定义扩展M函数**



Syslab



```
1 % Demonstrate synthesizeNTF
2 if exist('LiveDemo', 'var') == 0
3     LiveDemo=0;
4 end
5 format compact;
6 fig1pos1 = [9 630 200 200];
7 fig1pos2 = [10 407 480 420];
8 fig2pos1 = [239 630 450 200];
9 fig2pos2 = [241 341 523 485];
10
11 c1c
12 fprintf(1, '\t\tNTF Synthesis-- 5th-order modulator\n\n');
13 echo on
14 order = 5;
15 OSR = 32;
16 opt = 0;
17 H = synthesizeNTF(order, OSR, opt);
18 echo off
```



MWORKS.Syslab 202

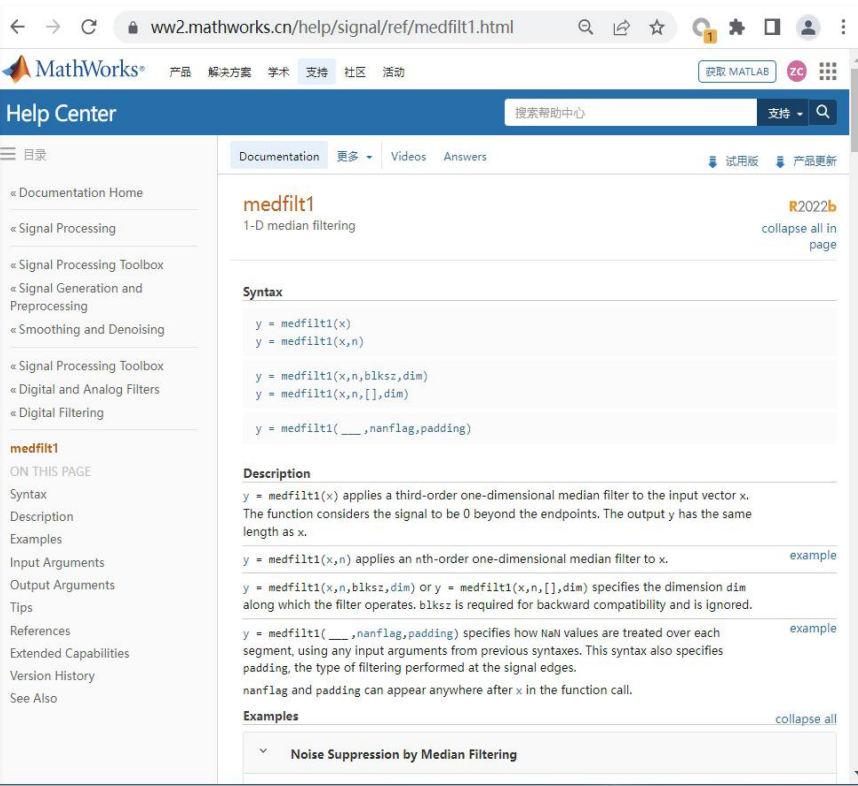
ExampleProject - Syslab

资源管理器 搜索 调试控制台 终端

1: Julia REPL (v1.7.3)

m1ang>

Julia env: v1.7



MathWorks

Help Center

Documentation Videos Answers

medfilt1

1-D median filtering

Syntax

```
y = medfilt1(x)
y = medfilt1(x,n)
y = medfilt1(x,n,blksz,dim)
y = medfilt1(x,n,[],dim)
y = medfilt1(__,nanflag,padding)
```

Description

y = medfilt1(x) applies a third-order one-dimensional median filter to the input vector x. The function considers the signal to be 0 beyond the endpoints. The output y has the same length as x.

Example

```
y = medfilt1(x,n) applies an nth-order one-dimensional median filter to x.
```

Example

```
y = medfilt1(x,n,blksz,dim) or y = medfilt1(x,n,[],dim) specifies the dimension dim along which the filter operates. blksz is required for backward compatibility and is ignored.
```

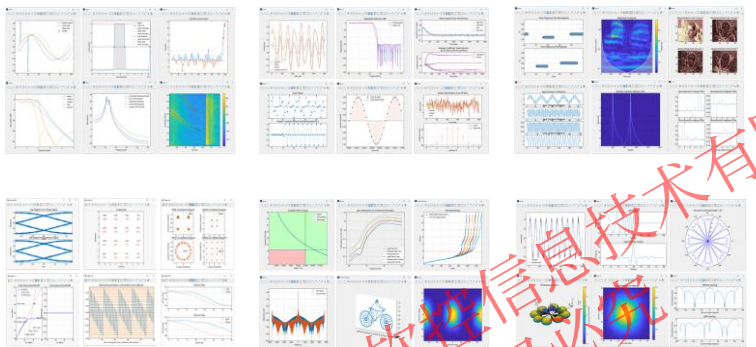
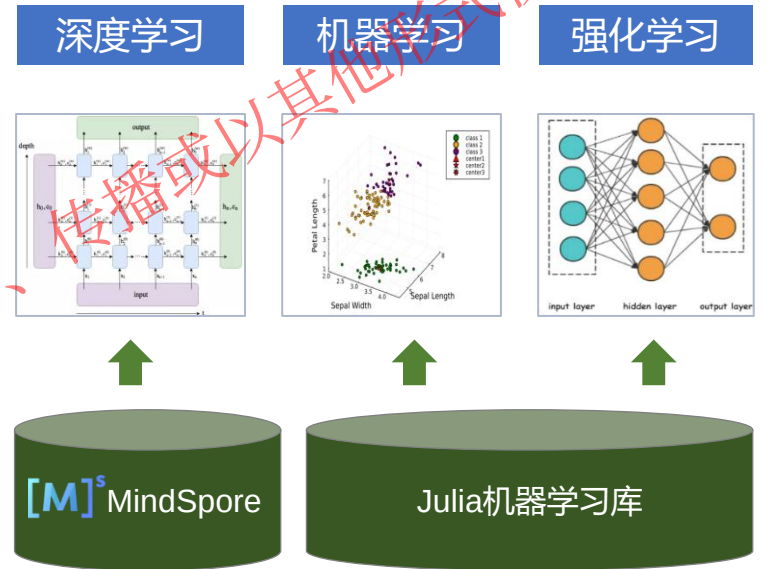
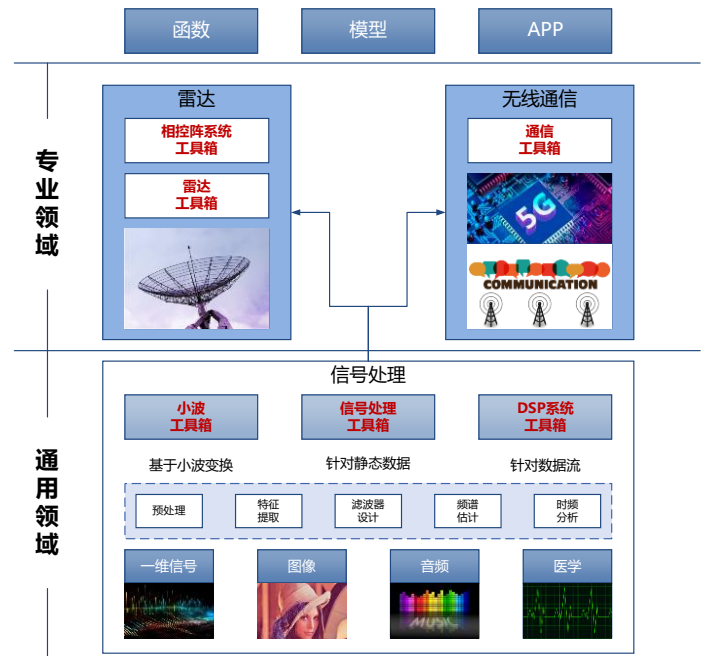
Example

```
y = medfilt1(__,nanflag,padding) specifies how NaN values are treated over each segment, using any input arguments from previous syntaxes. This syntax also specifies padding, the type of filtering performed at the signal edges.
```

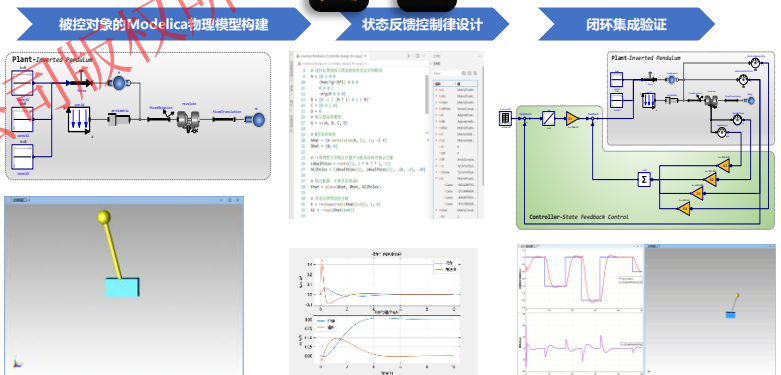
Examples

Noise Suppression by Median Filtering

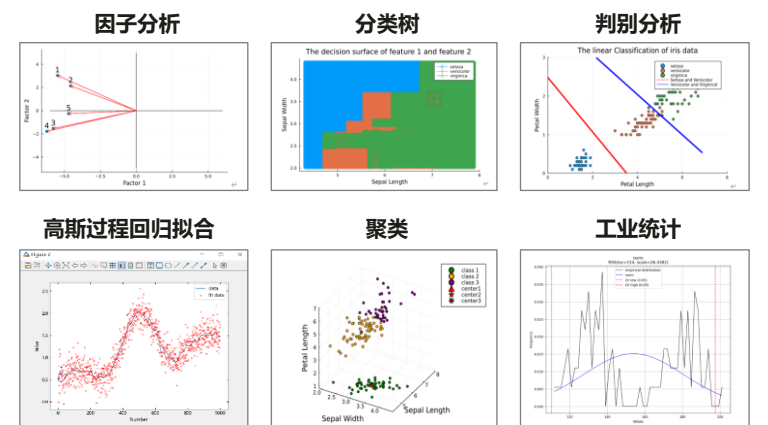
1.4 科学计算环境 Syslab – 专业系列工具箱



信号与通信系列工具箱



控制系列工具箱



AI与数据科学系列工具箱

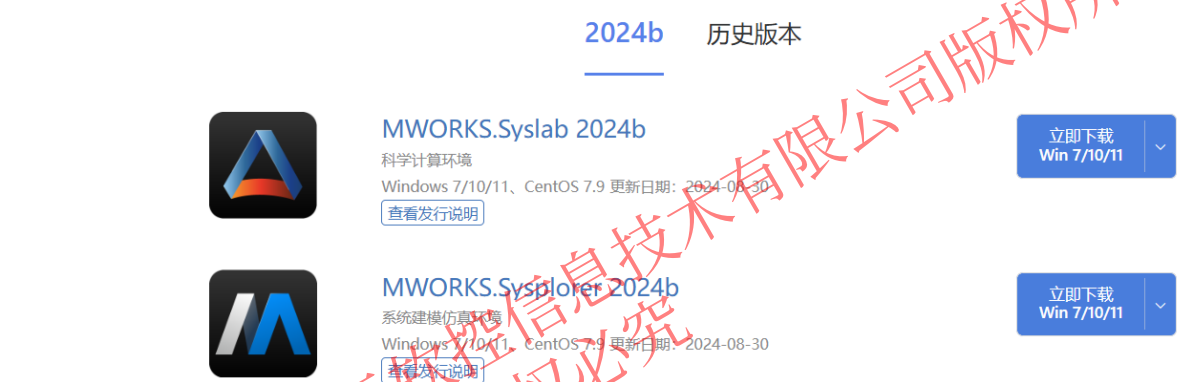
1.5 软件下载安装

软件下载地址：<https://www.tongyuan.cc/download>



安装说明:

1. 根据需求下载软件，两款软件可独立运行
2. 安装过程中需关闭杀毒软件或弹窗点击信任
3. 两款软件均可选择安装在任意磁盘



同元软控官网→技术支持→软件下载

1.5 软件下载安装

激活方式:

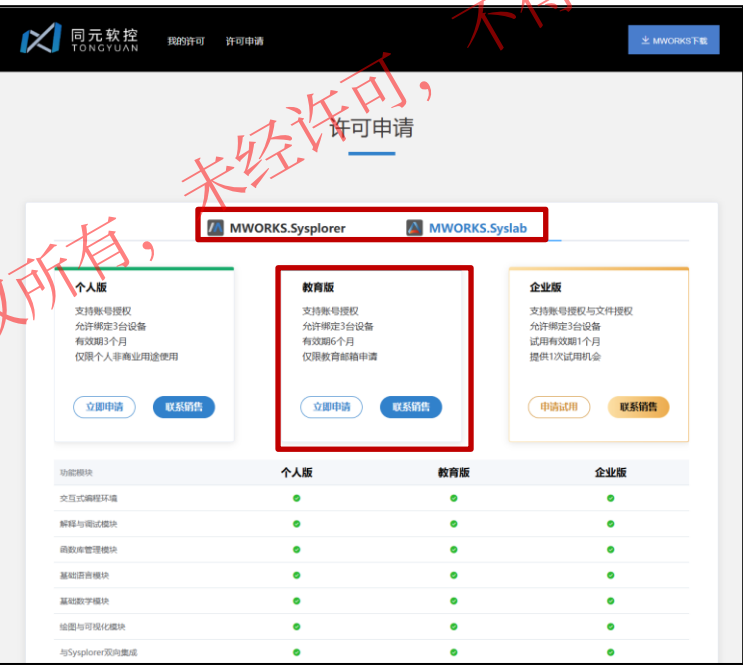
个人: 通过学校邮箱注册账号激活

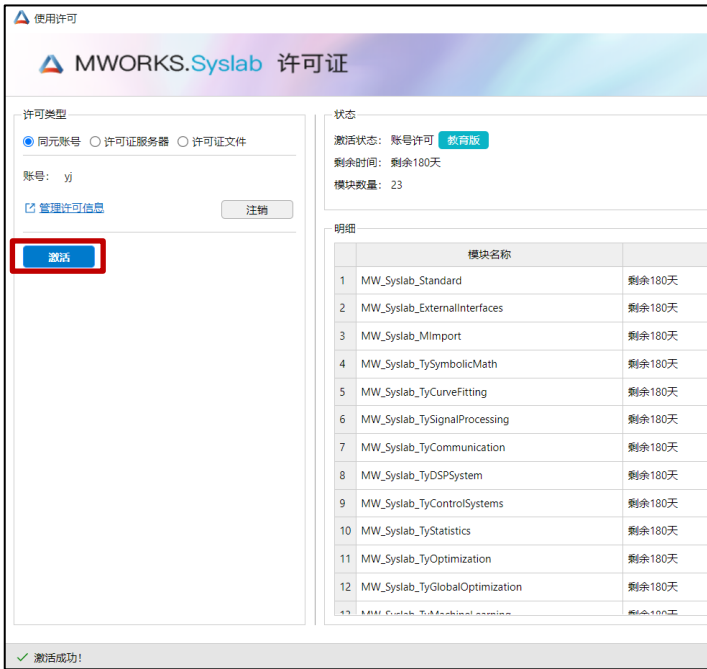
机房: 连接校园网通过服务器地址激活(部分高校已部署可用)











1. 通过edu邮箱注册账号

2. 申请对应软件的许可

3. 激活软件, 过期后可重复申请

1.5 软件下载安装

激活方式:

- 个人: 通过学校邮箱注册账号激活
- 机房: 连接校园网通过服务器地址激活(部分高校已部署可用)



说明:

1. 目前已有部分高校已部署校级服务器, 后续其他高校部署完成后会发主服务器地址至微信群和MoHub高校专区
2. 如老师机房着急上课, 可将授权服务部署至机房服务器

1. 输入IP地址点击激活即可

1.6 技术支持

MoHub技术交流/高校专区：提供大量常见问题FAQ，并安排专业工程师解答



高校专区：提供高校专属空间，促进校内互动与协作

MoHub网址：<https://mohub.net>

1.7 MWORKS相关资料

类别	材料
01-同元软控及MWORKS介绍	00-MWORKS产品介绍
	01-新一代科学计算与系统建模仿真平台MWORKS
	02-同元软控企业册
	03-同元软控标杆案例册
	04-同元软控航空案例册
	05-同元软控航天案例册
	06-同元软控能源案例册
	07-同元软控公司宣传片
	Logo
02-MWORKS产品功能列表	01-科学计算与系统建模仿真平台MWORKS-产品册
	02-MWORKS 用户手册
03-MWORKS与MATLAB功能对照	01-MWORKS与其他科学计算软件对比
	02-MWORKS与其他科学计算软件对比-录屏
04-同元基础培训材料	01-科学计算-培训材料
	02-系统建模仿真-培训材料
	03-培训材料列表-同元软控MWORKS
05-MWORKS应用推广培训	01-【虚拟教研室】师资培训系列讲座-2022
	02-【工信部人才交流中心】科学计算与系统建模仿真MWORKS培训-2023
	03-【北理工师资培训】科学计算与系统建模仿真MWORKS培训-2023
	04-【北航师资培训】科学计算与系统建模仿真MWORKS培训-2023
06-MWORKS.API	01-MWORKS.Sysplorer API详细说明
	02-MWORKS.Syslab API详细说明
07-MWORKS平台安装包	01-MWORKS.Sysplorer 安装包
	02-MWORKS.Syslab 安装包

获取方式:

百度网盘链接:

<https://pan.baidu.com/s/14sLpKBDDgNwUd89Lbtgpnw?pwd=TYRK>

提取码: TYRK

☐ 07-MWORKS平台安装包

☐ 06.MWORKS.API

☐ 05-MWORKS应用推广培训及课程替代研讨

☐ 04-同元基础培训材料

☐ 03-MWORKS与MATLAB功能对照

☐ 02-MWORKS产品功能列表

☐ 01-MWORKS平台介绍及典型案例

所有材料均为PDF版本

1.7 MWORKS相关资料

MoHub高校专区：提供教改内部资料和活动资讯



进入方式：

- 1. MoHub账号绑定**学校邮箱**直接进入
- 2. 若无学校邮箱可通过**邀请码**进入

- ☐ 07-MWORKS平台安装包
- ☐ 06.MWORKS.API
- ☐ 05-MWORKS应用推广培训及课程替代研讨
- ☐ 04-同元基础培训材料
- ☐ 03-MWORKS与MATLAB功能对照
- ☐ 02-MWORKS产品功能列表
- ☐ 01-MWORKS平台介绍及典型案例

所有材料均为PPT版本，可按需下载用于教学

MoHub网址：<https://mohub.net>

1.7 MWORKS相关资料

视频课程：提供各类软件使用视频课程，可供老师和学生自学



目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

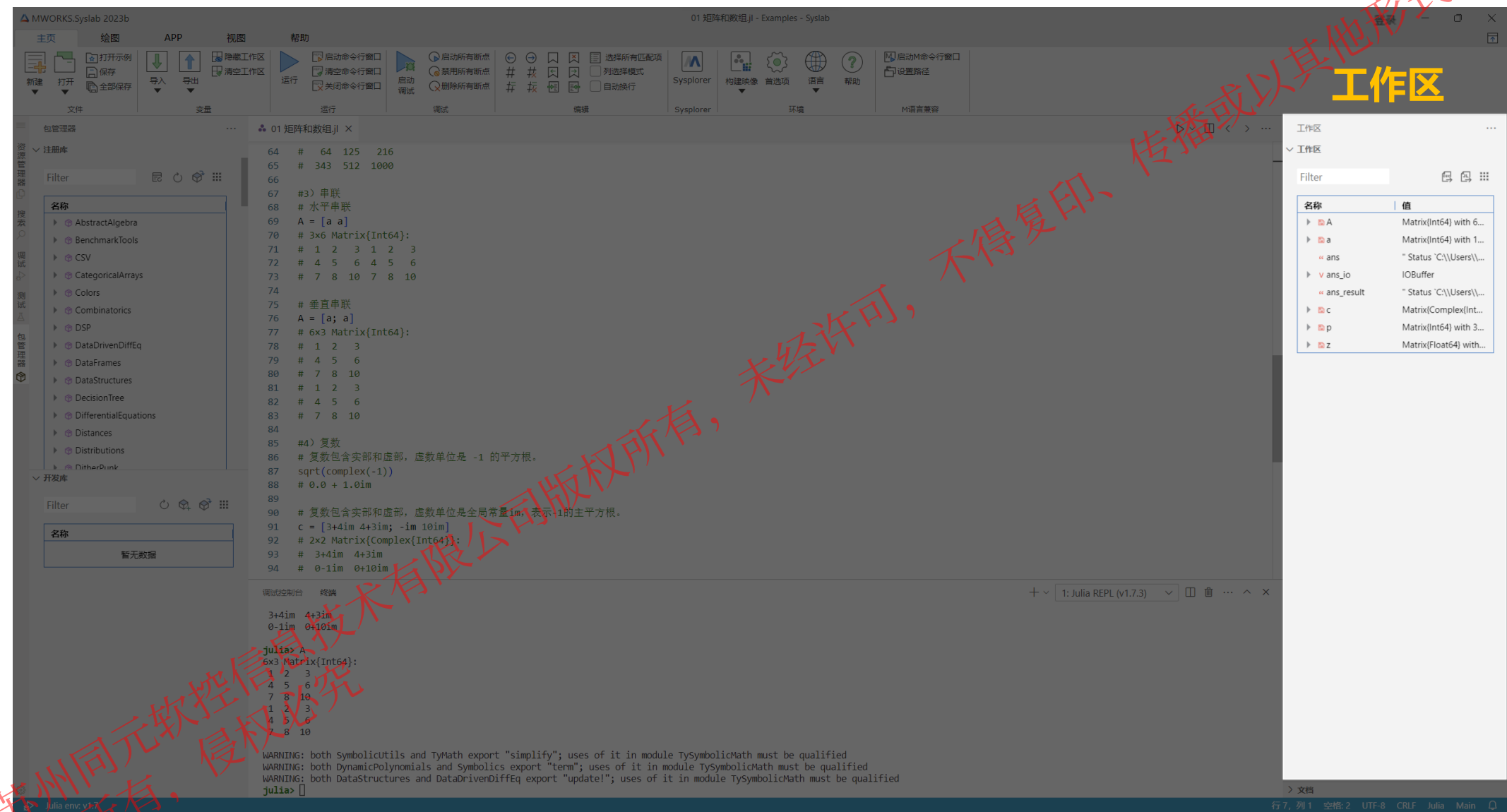
3. 工具箱使用

4. Julia脚本构建与调试

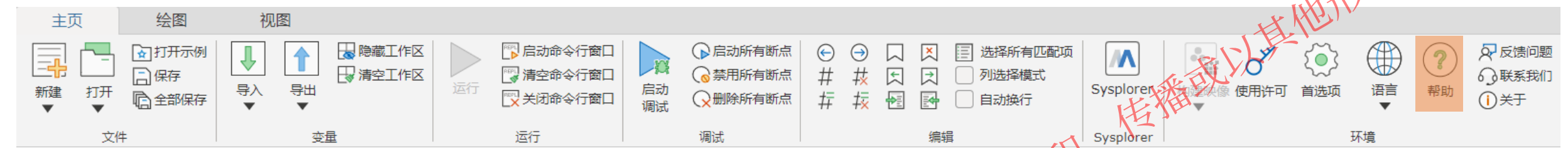
5. 结果后处理

6. 与其他语言的差异

2. MWORKS.Syslab软件界面简介



2. MWORKS.Syslab软件界面简介



- 1个基础环境
- 5个行业领域
- 3300+函数帮助
- 支持分类主题
(文档、示例、函数、App)
- 支持全局搜索
- 支持在线部署
- 支持集成用户帮助文档



Syslab帮助中心.mp4

目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

3. 工具箱使用

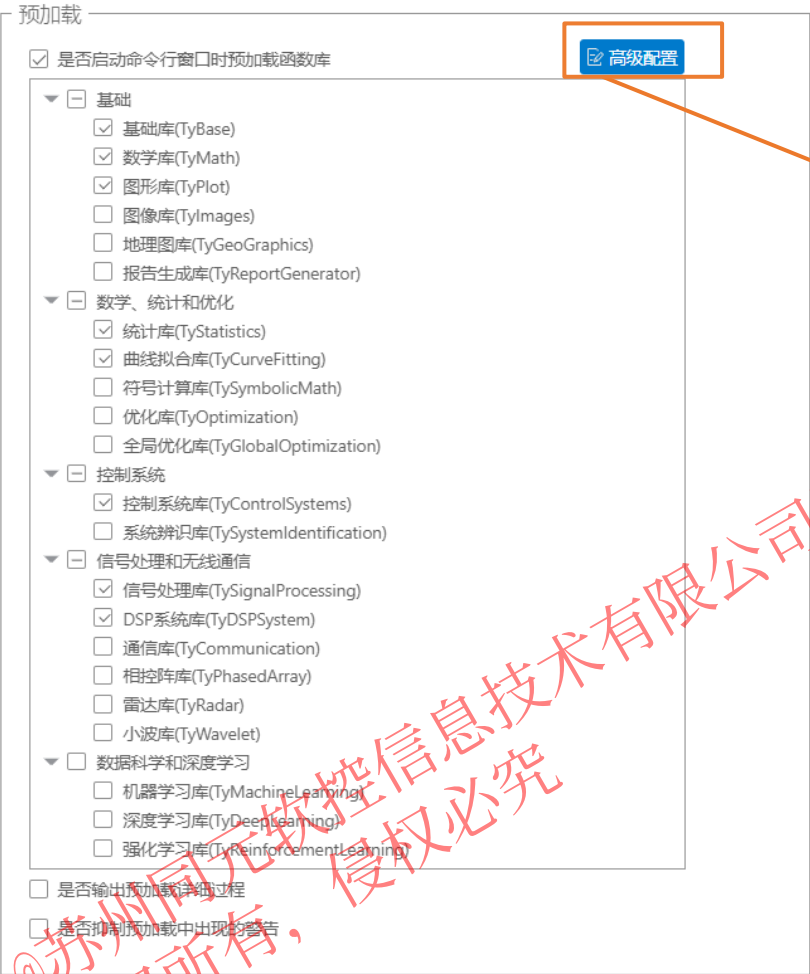
4. Julia脚本构建与调试

5. 结果后处理

6. 与其他语言的差异

3.1 工具箱使用

点击菜单栏中的  首选项，设置预加载同元商业工具箱，每次软件启动自动加载函数库。



```
# start_time = time_ns()

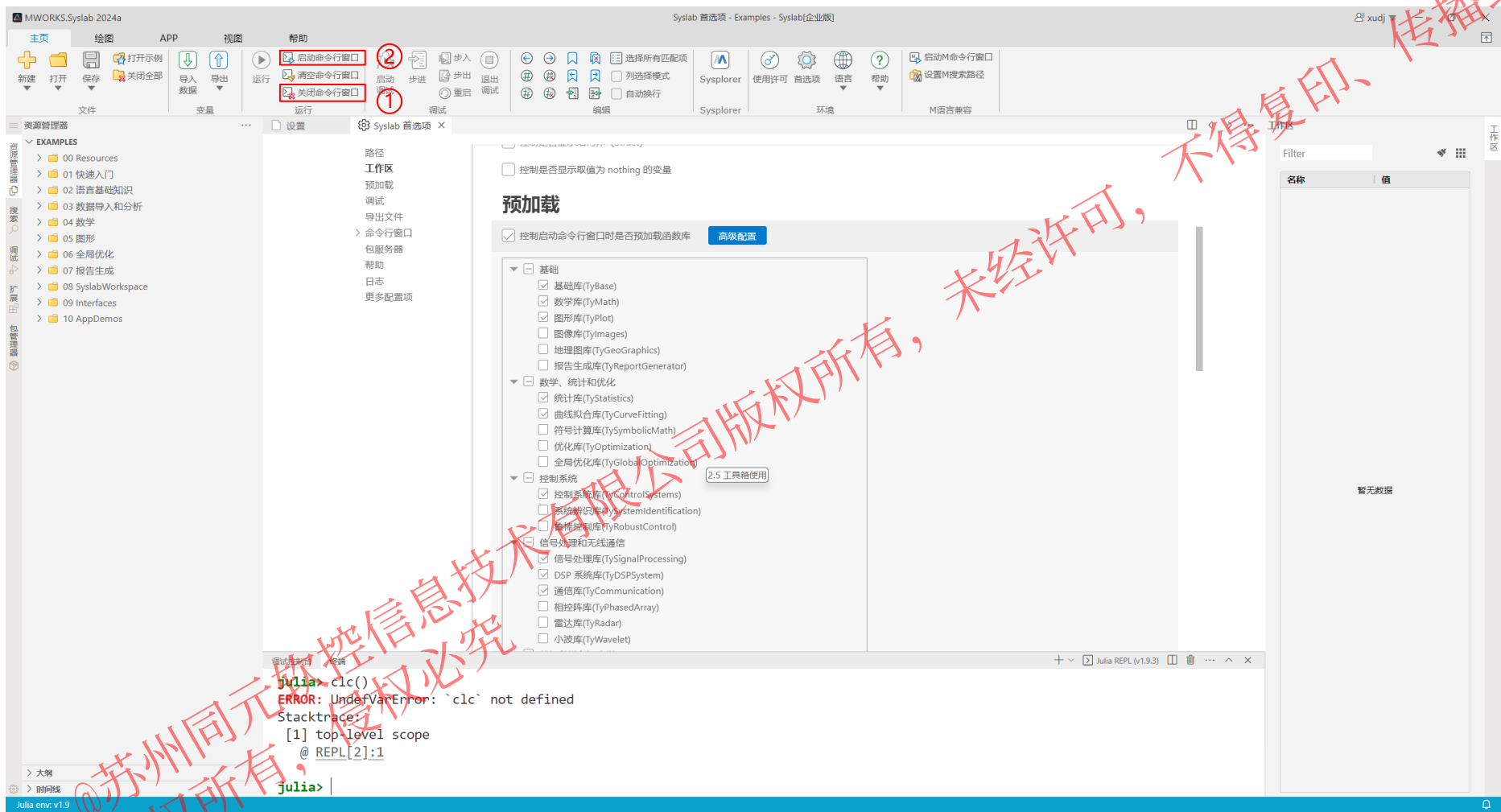
if haskey(ENV, "LANG") && startswith(ENV["LANG"], "zh")
    println("\n开始加载库，这可能需要一段时间...")
else
    println("\nStart loading packages, this may take a while...")
end

# 预编译
# import Pkg
# Pkg.precompile()

using Dates
using Printf
using Formatting
...
```

3.1 工具箱使用

Syslab安装后第一次打开软件默认不加载任何工具箱，按照上述加载方式加载工具箱后需重启命令行窗口进行工具箱加载。



- 1. 关闭命令行窗口
- 2. 重新启动命令行窗口



3.1 工具箱使用

在Julia中通过using使用已正确安装的第三方工具箱和同元商业工具箱，可直接调用使用的工具箱的函数。

```
using Flux, MLDatasets, Statistics
using Flux: onehotbatch, onecold, logitcrossentropy, Chain, params
using MLDatasets: MNIST
using Base.Iterators: partition
using Printf, BSON
using Parameters: @with_kw
using CUDA
```

```
model = sum(A) #直接调用已预加载工具箱中的函数sum对数组A求和
```

using TyMathCore

using MatrixPencils

```
julia> spzeros
WARNING: both MatrixPencils and TyMathCore export "spzeros"; uses of it in module Main must be qualified
ERROR: UndefVarError: `spzeros` not defined

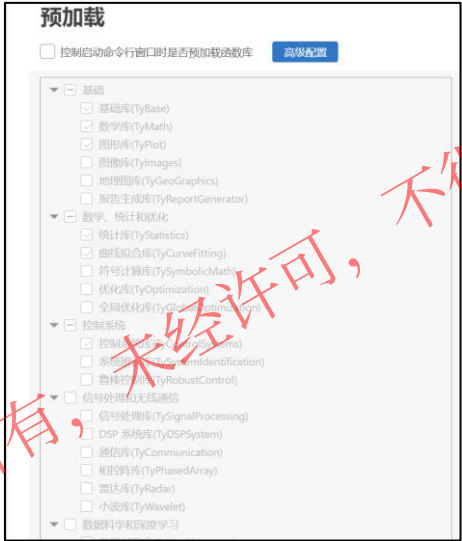
julia> MatrixPencils.spzeros
spzeros (generic function with 1 method)
```

注：当多个库中的函数产生冲突时，需要使用库名+函数名的方式

3.2 图形工具箱plot函数使用为例

将 x 创建为由 0 和 2π 之间的线性间隔值组成的向量。在各值之间使用递增量 $\pi/100$ 。将 y 创建为 x 的正弦值。创建数据的线图。

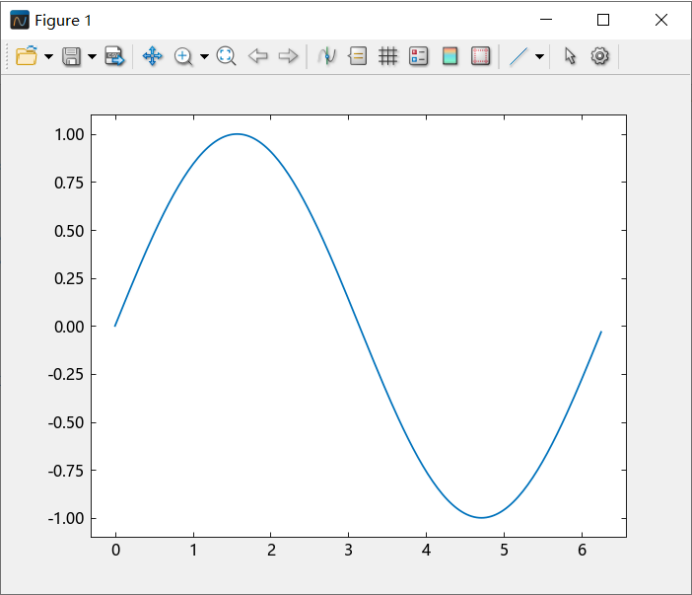
```
#using TyPlot
x = 0:(pi / 100):(2 * pi);
y = sin.(x);
plot(x, y)
```



ERROR: UnderVarError: `plot` not defined
Stacktrace:
[1] top-level scope
@ Untitled-2.jl:4

- 1.模型库加载
- 2.函数调用

```
using TyPlot
x = 0:(pi / 100):(2 * pi);
y = sin.(x);
plot(x, y)
```



目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

3. 工具箱使用

4. Julia脚本构建与调试

5. 结果后处理

6. 与其他语言的差异

4.1 交互式编程环境

操作步骤:

- 在julia>后输入命令，敲击“Enter”，Syslab自动执行并得到执行结果
- 定义变量、函数、执行循环语句

```
julia> a = "Hello, World" # 定义变量
"Hello, World"

julia> a # 展示变量
Hello, World

julia> function f(x, y) # 定义函数
    return x + y
end

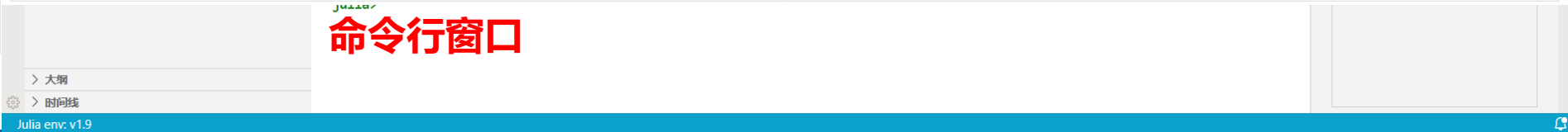
julia> for i = 1:10 # 执行循环语句
    a = i * 2
    print(a)
end
```

基本命令

操作	命令
上一次运算的结果	ans
中断命令执行	Ctrl+C
清屏	Ctrl+L
运行程序文件	include("filename.jl")
查找 func 相关的帮助	?func
查找 func 的所有定义	apropos("func")
命令行模式	;
包管理模式	]
帮助模式	?
查找特殊符号输入方式	?☆ # "☆" can be typed by \bigwhitestar<tab>
退出特殊模式返回到命令行窗口	在空行上按回格键[Backspace]
退出命令行窗口	exit() 或 Ctrl+D

命令行窗口常用基本命令

命令行窗口



4.1 交互式编程环境

高效的使用 Julia REPL

基于历史回滚输入：

Julia REPL可以基于已经输入的内容回滚历史输入（快捷键：上方向键），用户可以通过该特性高效的查找历史输入。

```
julia>function compute_area(length, width)
    area = length + width
    println("The area is $area")
end
```

以前定义的函数存在错误

```
julia>function compute_area(length, width)
    area = length * width
    println("The area is $area")
end
```

通过**上方向键**重现该函数，并将光标移至出错的位置对其进行修改

这些历史命令在关闭 REPL 后依然会保留，因此，即使用户退出了 REPL，历史输入也可以在下次启动时被查找到。

4.1 交互式编程环境

高效的使用 Julia REPL

使用 TAB 进行自动补全:

在 REPL 中可以输入一个函数或类型的前几个字符，然后按下 Tab 键触发自动补全功能，当 REPL 检测到多种补全项后，将展示所有匹配项的列表:

```
julia> test = 1 # 定义名为 test 的变量
1

julia> te[按下TAB] # 展示当前作用域中所有由 te 开头的变量和类型
tempdir      tempname      test          text"          textwidth

julia> tes[按下TAB] # 当前作用域中只有变量 test 由 tes 开头，REPL将自动补全输入
为 test
julia> test
```

4.1 文件管理

打开相关文件夹作为工作目录，并在资源管理器中管理文件。

操作步骤:

- 选择工作目录：菜单栏中“打开”相关文件夹或文件
- 新建脚本文件-Test
 - 工作目录下新建脚本
 - 工作目录下新建文件夹
- 对文件进行重命名
- 不需要的脚本可以delete或右击选择删除
- “新建”选择文件格式

Tips: 新建Python或m脚本，Syslab可以使用Python或m环境，Syslab内置Python环境版本为3.7.5

运行当前文件

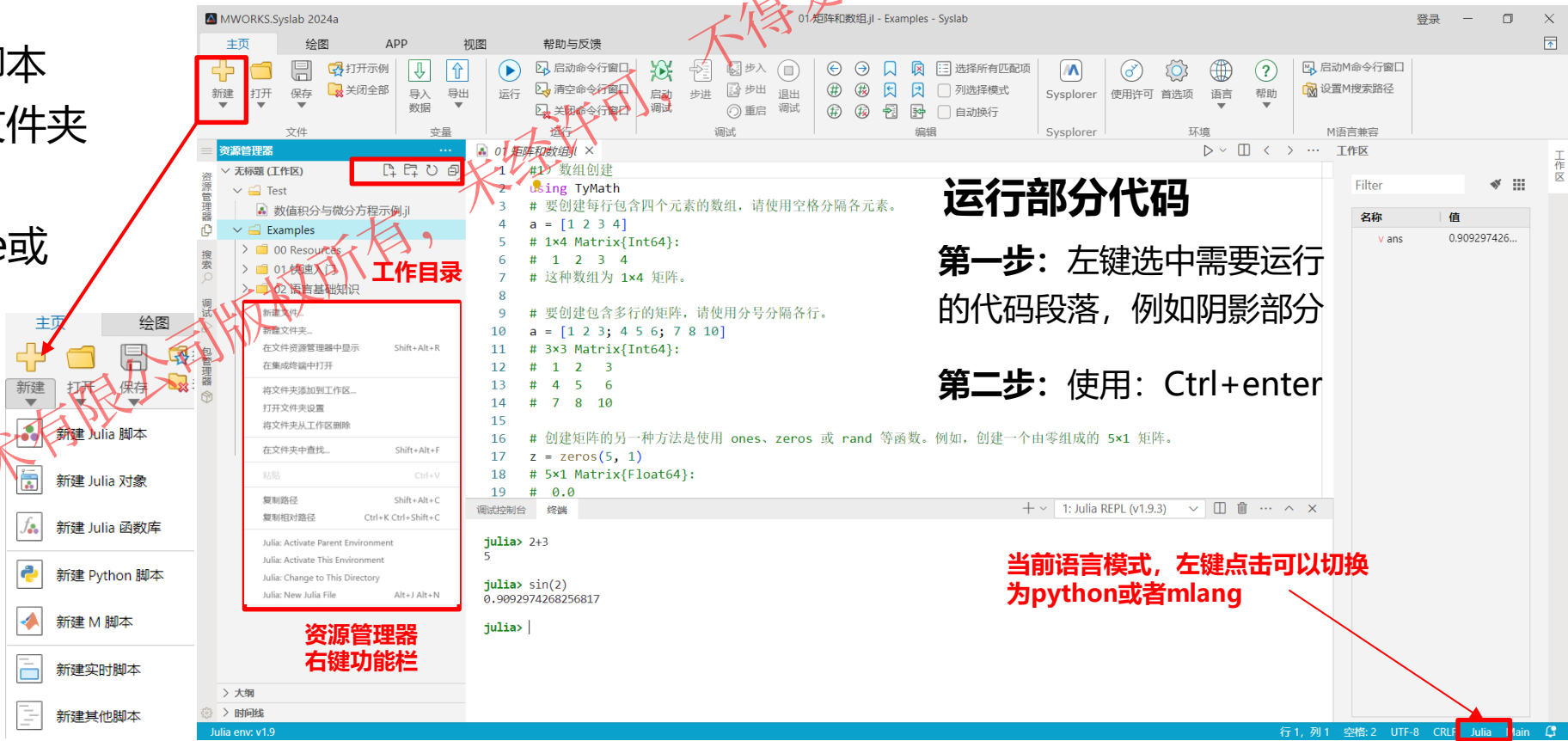
- 主页工具栏运行按钮
- 快捷键：Ctrl+F5

运行部分代码

第一步：左键选中需要运行的代码段落，例如阴影部分

第二步：使用：Ctrl+enter

当前语言模式，左键点击可以切换为python或者mlang



4.1 Julia脚本构建-变量

变量是与某个值相关联（或绑定）的名字，可以用它来保存一个值（例如某些计算得到的结果），供之后的代码使用。可以直接在命令行窗口中创建变量。

例：创建变量a、b、c，分别赋值为5、10.5和“HelloWorld”，并查看变量的类型。

```
julia> a = 5
```

```
5
```

一般形式: `x = val`

```
julia> b = 10.5
```

```
10.5
```

```
julia> c = "Hello World"
```

```
"Hello World"
```

```
julia> typeof(a)
```

```
Int64
```

一般形式: `typeof(x)`

```
julia> typeof(b)
```

```
Float64
```

```
julia> typeof(c)
```

```
String
```

变量名为英文，规则如下：

- 区分大小写；
- 不能以数字开头；
- 变量与函数名建议用下划线分隔；
- 类与模块首字母建议大写，驼峰式；

变量的数据类型由赋值决定

4.1 Julia脚本构建-变量

变量不只可以为英文，还可以是中文或特殊字符

例：创建变量“变量1”，值为3.14，并查看变量的值。

```
julia> 变量1 = 3.14
3.14

julia> 变量1
3.14
```

例：创建变量“β”，值为36，并查看变量的值。

```
julia> β = 36 #\beta,再敲击Tab键
36

julia> β
36
```

Unicode字符使用方式：
输入某个LaTeX符号（比如\beta），再敲击Tab键。

LaTeX符号详见：《Julia 中文文档》- Unicode输入表



4.1 Julia脚本构建-数值运算

可以对变量直接使用运算符进行计算

例：计算梯形面积，其中梯形上下底和高分别为10、30和15。

```
julia> (10+30)*15/2
300.0
```

直接计算面积
计算结果只能下一个语句使用

```
julia> a = 10
10

julia> b = 30
30

julia> h = 15
15

julia> s = (a+b)*h/2
300.0
```

定义变量计算面积
定义的变量可多次重复使用

```
julia> a = 10;
julia> a=10;b=30;h=15;s=(a+b)*h/2
300.0
```

命令结尾使用`;`则此命令行结果不显示
使用`;`可以将多句命令合并为一句



4.1 Julia脚本构建-数值运算

Syslab中提供了大量数值运算符，可以直接使用

类型	表达式	名称	描述
算数运算符	+x	一元加法	全等操作
	-x	一元减法	将值变为其相反数
	$\sqrt{x}$	平方根	求解平方根(\sqrt{x})
	x + y	二元加法	执行加法
	x - y	二元减法	执行减法
	x * y	乘法	执行乘法
	x / y	除法	执行除法
	x $\div$ y	整除	取x/y的整数部分(\div)
	x \ y	反向除法	等价于y/x
	x ^ y	幂运算	x的y次幂
	x % y	取余	等价于rem(x,y)



4.1 Julia脚本构建-数组

向量定义

```
julia> a = [1,2,3]
3-element Vector{Int64}:
 1
 2
 3
```

一般形式: **x = [x1,x2,x3]**

使用 **,** 分隔向量的元素

```
julia> b = [1;2;3]
3-element Vector{Int64}:
 1
 2
 3
```

一般形式: **x = [x1;x2;x3]**

使用 **;** 分隔向量的元素

矩阵定义

```
julia> c = [1 2;3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

一般形式: **x = [x1 x2;x3 x4]**

使用 **空格** 分隔矩阵列, 使用 **;** 分隔矩阵行

```
julia> c = [1 2
            3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

一般形式: **x = [x1 x2
x3 x4]**

使用 **空格** 分隔矩阵列, 使用 **Alt+Enter** 分隔矩阵行



4.1 Julia脚本构建-数组

函数	功能
ones	生成全1元素数组
zeros	生成全0元素数组
eye	生成单位矩阵
rand	生成均匀分布随机矩阵
randn	生成正态分布随机矩阵
magic	生成魔方矩阵，即矩阵各行各列的和都相等
Diagonal	生成对角矩阵

特殊数组内置函数

构建向量a[3]，其中a每个元素均为1

```
julia> a = ones(3)
3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

构建3阶单位矩阵b

```
julia> b = eye(3)
3×3 Matrix{Int64}:
 1 0 0
 0 1 0
 0 0 1
```

```
julia> b = eye(3,3)
3×3 Matrix{Int64}:
 1 0 0
 0 1 0
 0 0 1
```



4.1 Julia脚本构建-数组

向量拼接为矩阵

定义3阶0向量a,
定义3阶1向量b,
将a向量和b向量拼接形成矩阵A

```
julia> a = zeros(3);  
  
julia> b = ones(3);  
  
julia> A = [a b]  
3×2 Matrix{Float64}:  
0.0 1.0  
0.0 1.0  
0.0 1.0
```

矩阵拼接为矩阵

将3阶魔方矩阵A拼接形成矩阵B:
$$\begin{bmatrix} A & \text{zeros}(3,1) \\ \text{zeros}(1,3) & 1 \end{bmatrix}$$

```
julia> A = magic(3);  
  
julia> B = [A zeros(3,1);zeros(1,3) 1 ]  
4×4 Matrix{Float64}:  
8.0 1.0 6.0 0.0  
3.0 5.0 7.0 0.0  
4.0 9.0 2.0 0.0  
0.0 0.0 0.0 1.0
```

- 分号表示纵向(行, 第一维度)拼接
- 空格表示横向(列, 第二维度)拼接
- 空格的优先级高于分号, 即先横向拼接, 再纵向拼接



4.1 Julia脚本构建-数组

定义3阶魔方矩阵A,
访问A的第1行, 第2列的元素,
访问A的第1行所有元素,
访问A的第2列所有元素,
访问A的第1行至第2行, 第2列到第3列所有元素,
访问A的第2行到最后1行的所有元素。



访问A的第m行第n列的元素, $a = A[m,n]$
访问A的第m行的所有元素, $b = A[m,:]$
访问A的第n列的所有元素, $c = A[:,n]$
访问A的第n行到第m行, 第n列到第m列元素, $d = A[n:m,n:m]$
访问A的第m行到最后一行的所有元素, $e = A[m:end,:]$

```
julia> A = magic(3)
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2
```

```
julia> B = A[1,2]
1
```

```
julia> C = A[1,:]
3-element Vector{Int64}:
 8
 1
 6
```

```
julia> D = A[:,2]
3-element Vector{Int64}:
 1
 5
 9
```

```
julia> E = A[1:2,2:3]
2×2 Matrix{Int64}:
 1  6
 5  7
```

```
julia> F = A[2:end,:]
2×3 Matrix{Int64}:
 3  5  7
 4  9  2
```

4.1 Julia脚本构建-数组

常用运算符与函数

类型	表达式	名称	描述
一元运算符	+	一元加法	全等操作
	-	一元减法	将各元素取反
二元运算符	-	减	各元素相减
	+	加	各元素相减
	.*	点乘	各元素相乘
	./	点除	各元素相除
	*	乘	数组乘法（线性代数）
	/	除	数组除法（线性代数）
	\	反向除法	数组反向除法（线性代数）
	^	幂	数组幂运算
常用函数	det	矩阵行列式	计算矩阵行列式
	rank	矩阵的秩	计算矩阵的秩
	inv	矩阵求逆	计算矩阵的逆
	eigvals	矩阵的特征值	计算矩阵的特征值
	transpose	向量或矩阵转置	将向量或矩阵进行转置



4.1 Julia脚本构建-函数

Syslab中集成了大量内置函数，可根据需求直接使用，并提供了丰富的中文帮助说明

计算sin(2)，cos(3)，并取sin(2)和cos(3)的最大值和最小值

```
julia> a = sin(2)
0.9092974268256817

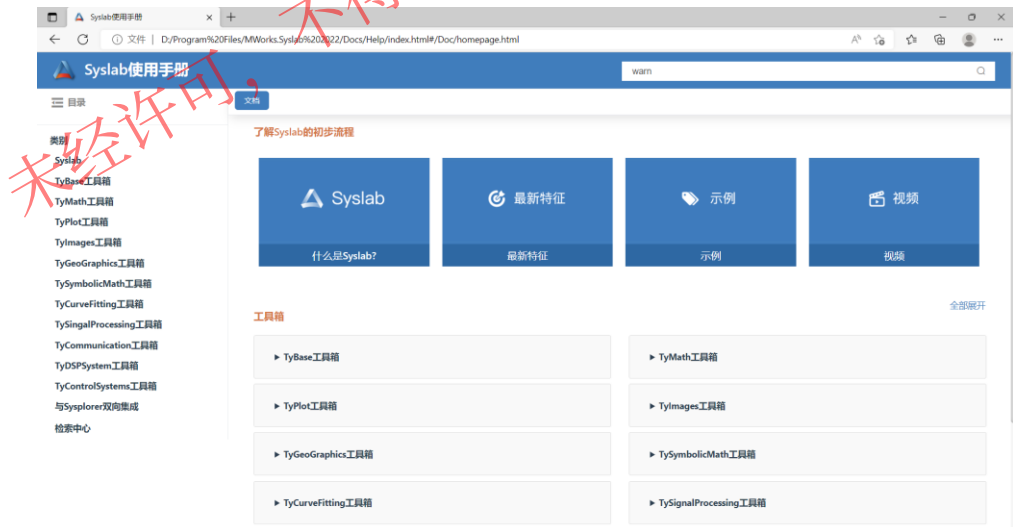
julia> b = cos(3)
-0.9899924966004454

julia> c = max(a,b)
0.9092974268256817

julia> d = min(a,b)
-0.9899924966004454
```

使用形式: `x = func(a,b,c)`

内置函数的功能、使用方式、示例均提供帮助说明
方式1: 可查看Syslab帮助中心



方式2:
Shift+? 进入help模式
输入函数名即可获得帮助说明

```
help?> plot
search: plot plot3 plotmatrix plot_orb

plot - 二维线图

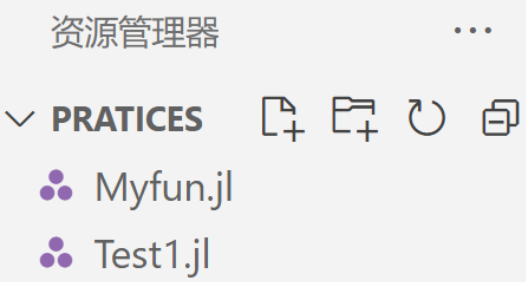
此 Syslab 函数 创建 Y 中数据对 X 中X
则它们的大小必须相同。plot 函数绘制 Y
中的一个为向量而另一个是矩阵，则矩阵
是或向量。则 plot 函数会绘制离散点，但
```



4.1 Julia脚本构建-函数

当内置函数不能满足需求时，可根据需求自定义函数，并进行使用

定义函数 $f(x_1, x_2) = x_1^2 + x_2^2 - 2x_1x_2 + 6x_1 - 6x_2$ 并求解 $f(1,2)$ 的值



新建脚本文件Myfun.jl

```
function myfun(x1,x2)
z = x1^2-2x1*x2+6x1+x2^2-6x2;
end
```

定义函数myfun
x1和x2为形参，z为返回值

```
myfun (generic function with 1 method)
```

运行函数定义

```
julia> a = myfun(1,2)
-5
```

函数调用

```
myfun(x1, x2) = x1^2 - 2x1 * x2
+ 6x1 + x2^2 - 6x2
```

简写定义函数myfun
x1和x2为形参

```
include("Myfun.jl")
a = myfun(1,2)
```

脚本中直接调用函数脚本
include("函数脚本路径")



4.1 Julia脚本构建-函数

任何函数f均能够以元素方式作用于任何数组（或者其它集合），这通过语法**f.(A)**实现。

计算 $z_2 = \frac{1}{2} \ln(x + \sqrt{1+x^2})$ ，其中 $x = \begin{bmatrix} 2 & 1+2i \\ -0.45 & 5 \end{bmatrix}$

```
julia> x = [2 1+2im;-0.45 5]
2×2 Matrix{ComplexF64}:
 2.0+0.0im  1.0+2.0im
-0.45+0.0im  5.0+0.0im

julia> z2 = 1/2*log.(x+sqrt.(1.+x.^2))
2×2 Matrix{ComplexF64}:
0.711397-0.0253138im  0.896799+0.365773im
0.21387+0.934284im   1.15408-0.00442594im
```

复数构造形式: **a + b* im**

sqrt.(A)即表示对数组A中的每一个元素进行求平方根



4.1 Julia脚本构建-控制流

一般格式:

```
if 条件表达式1
    #执行语句
elseif 条件表达式2
    #执行语句
elseif 条件表达式3
    #执行语句
else
    #执行语句
end
```

注意:

- 条件表达式中if是必须的, 且只能出现一次;
- else可以不出现但若出现则只能一次;
- elseif可多次使用;
- 条件表达式与关键词之间用空格隔开
- 条件表达式为布尔量

```
julia> if 3>0
    3
elseif 2>0
    2
end
```

3

```
julia> if 1
    println("true")
end
```

ERROR: TypeError: non-boolean (Int64) used in boolean context

```
julia> z = if 3 < 0
    10
else
    -10
end
```

-10

4.1 Julia脚本构建-控制流

一般格式:

```
while 条件表达式
    #执行语句
end
```

```
julia> i=1
1

julia> while i <= 5
    println(i)
    i += 1
end

1
2
3
4
5

julia> i
6
```

说明:

- 条件表达式为布尔量，为true时，执行语句才会执行;



4.1 Julia脚本构建-控制流

一般格式(单个迭代变量):

格式1:

```
for 迭代变量=可迭代数集
    #语句块
end
```

格式2:

```
for 迭代变量 in 可迭代数集
    #语句块
end
```

格式3:

```
for 迭代变量 ∈ 可迭代数集
    #语句块
end
```

注意:

- 三个遍历操作符=、in及∈是等价的，可以任选一个；
- ∈输入：\in + tab键
- 可迭代的内容包括元组、字典、集合、数组、及列表等

```
julia> for i = 1:5
    print(i)
end
12345

julia> for i in [1 2 3 4 5]
    print(i)
end
12345

julia> for i ∈ (1, 2, 3, 4, 5)
    print(i)
end
12345
```

4.1 Julia脚本构建-控制流

多个迭代变量:

```
julia> for i = 1:2, j in (1, 2, 3)
    print((i, j))
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)

julia> for i in [1 2 3], j = 3:4
    print((i, j))
end
(1, 3)(1, 4)(2, 3)(2, 4)(3, 3)(3, 4)

julia> for j = 3:4, i in [1 2 3]
    print((i, j))
end
(1, 3)(2, 3)(3, 3)(1, 4)(2, 4)(3, 4)
```



```
julia> for i = 1:2
    for j in (1, 2, 3)
        print((i, j))
    end
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```

```
julia> for i=1:2;j=3:4
    print((i,j))
end
(1, 3:4)(2, 3:4)
```

说明:

- for循环支持同时对多个数据集进行遍历;
- 遍历操作符的顺序影响内部语句循环变量的取值。

注意:

- 每一个遍历表达式之间的分割符是逗号而非分号, 当为分号时, 这种表达式将为复合表达式而非遍历表达式。



4.1 Julia脚本构建-控制流

break—强制退出循环

```
julia> i=1
1

julia> while i < 10
    if i > 5
        break
    end
    print(i)
    i += 1
end
12345
```

说明:

- break强制循环在满足控制条件时退出。
- break在for循环中的使用与while循环相同。

continue—强制跳过当前迭代

```
julia> i=1
1

julia> while i < 10
    i += 1
    if i < 5
        continue
    end
    print(i)
end
5678910

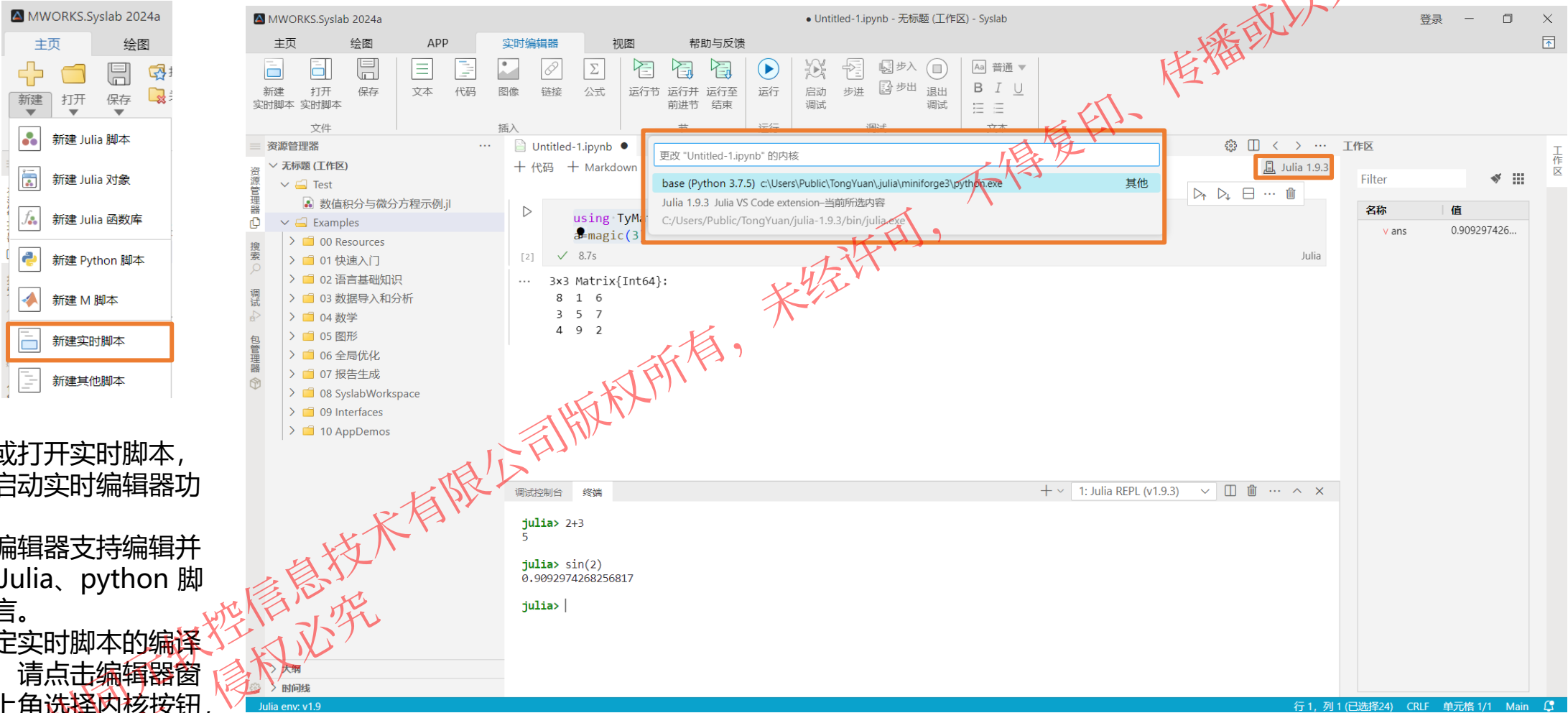
julia> i
10
```

说明:

- continue可以让循环跳过当前迭代，直接进入下一个循环的迭代。
- continue在for循环中的使用与while循环相同。

4.1 实时编辑器

支持用户在统一的文档环境中将代码与嵌入式输出、格式化文本、方程和图像组合到一起，生成可交互式记事本，并与他人分享。



- 注意:
- 新建或打开实时脚本，才会启动实时编辑器功能
 - 实时编辑器支持编辑并运行 Julia、python 脚本语言。
 - 要指定实时脚本的编译内核，请点击编辑器窗口右上角选择内核按钮，选择要使用的语言内核。

Tips: 此处的Julia脚本默认不加载任何工具箱，用户需要自己using XXX函数库，包括同元函数库

4.1 实时编辑器

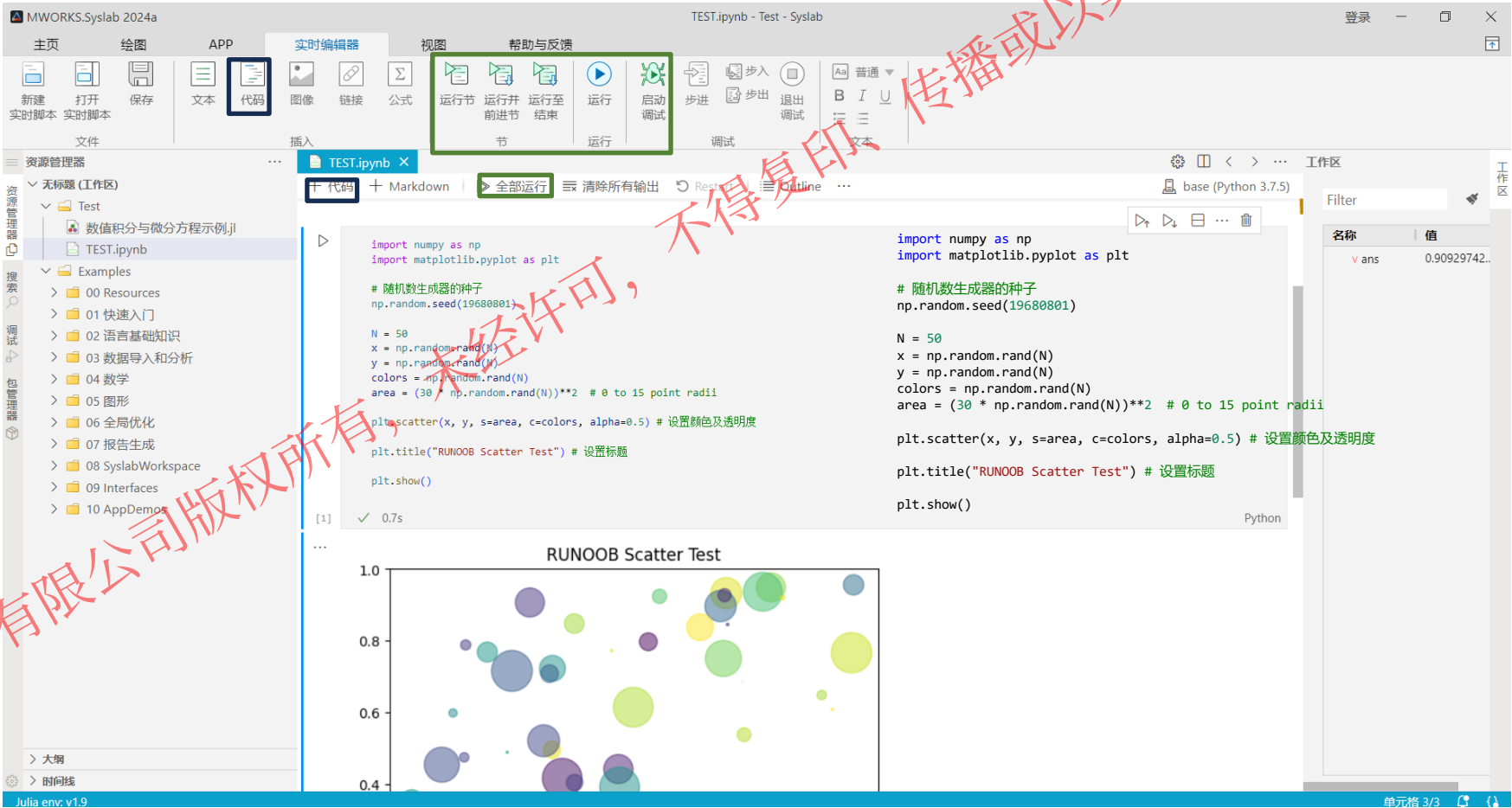
可新建并编辑Julia/Python代码，并进行运行和调试

新建Julia/ Python代码：

- 方式1：
- 方式2：+ 代码

代码运行：

- 运行节：运行当前节
- 运行并前进节：运行当前节并前进到下一节
- 运行至结束：从当前节运行至代码结束
- 运行或全部运行：运行全部代码
- 停止：终止运行进程
- 调试见《脚本调试》章节

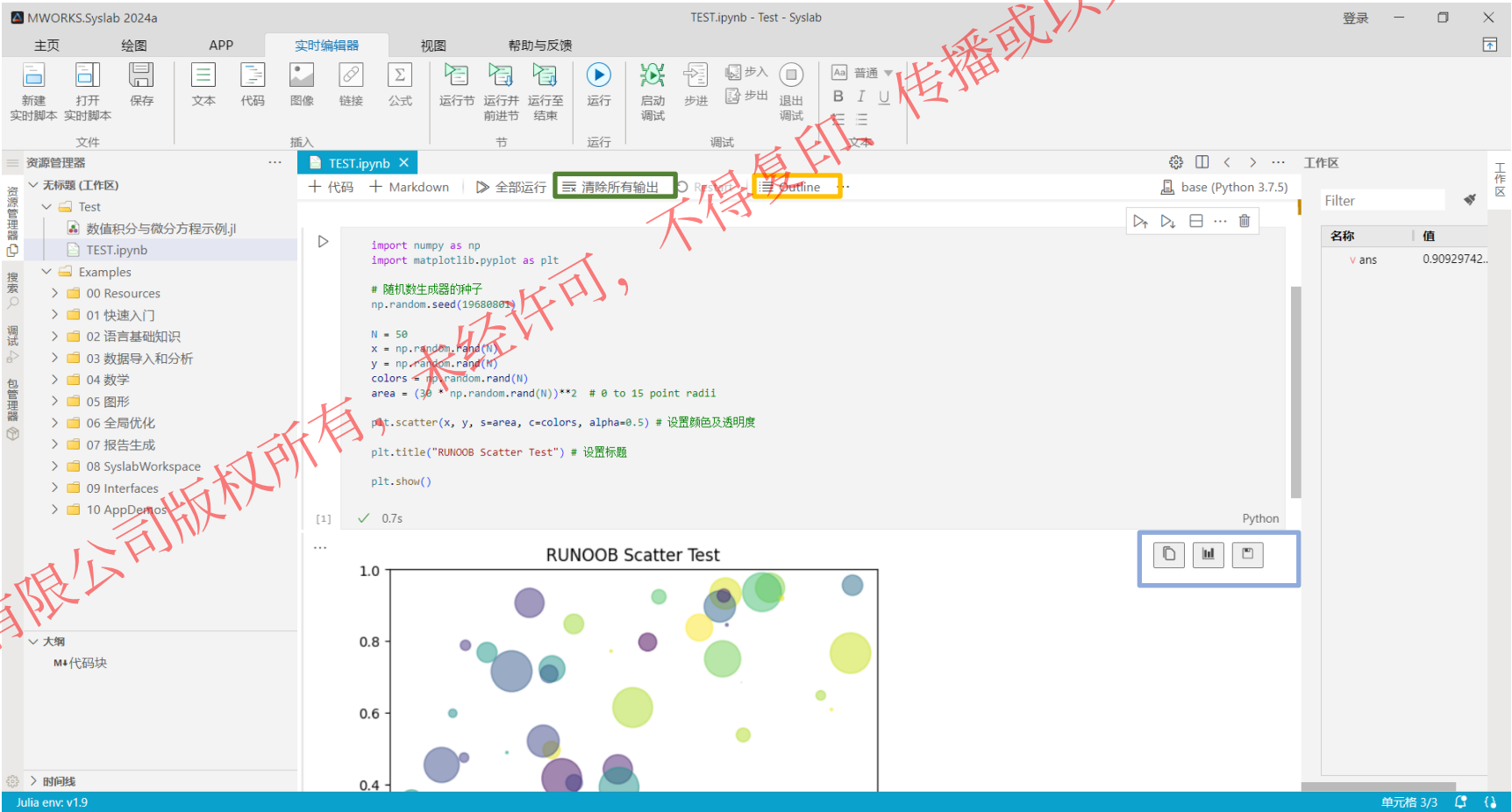


4.1 实时编辑器

可对运行结果的变量和图形进行查看并导出

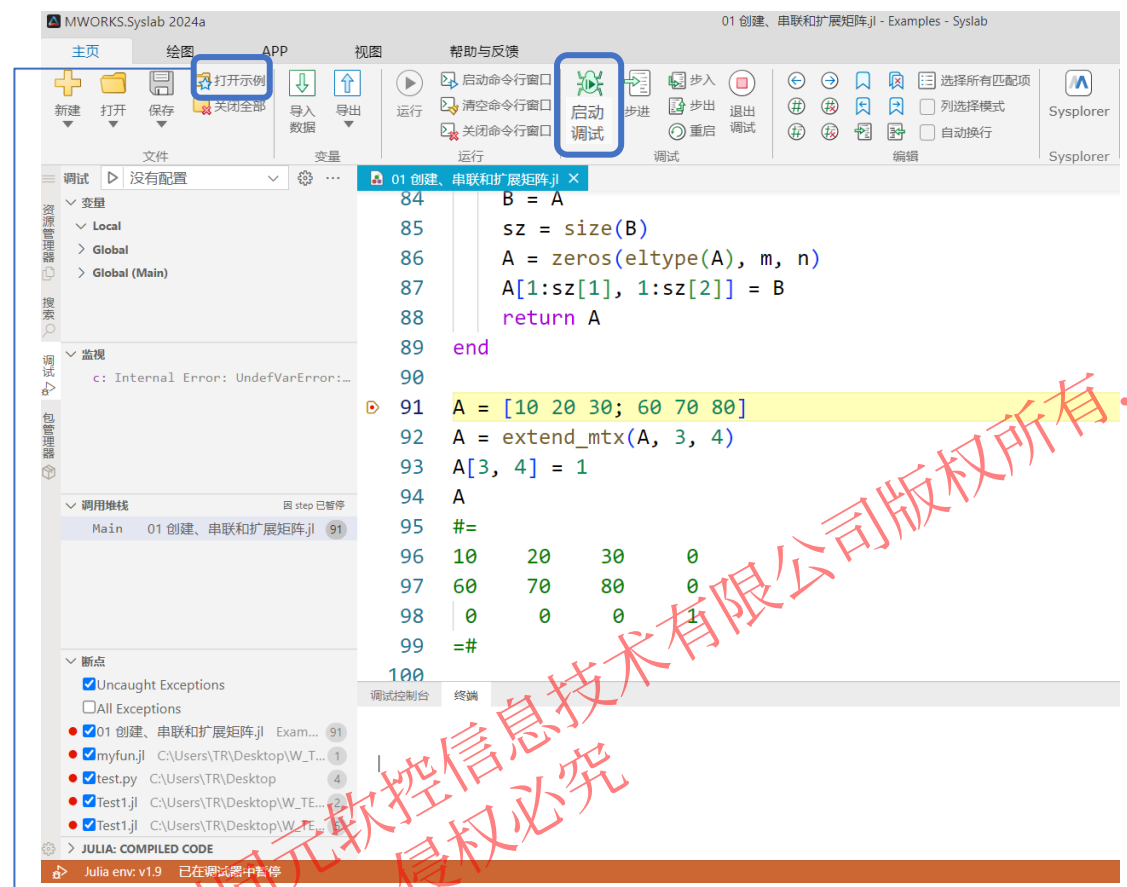
说明:

- 运行结束后，可在代码块下方直接查看图形结果
-  清除所有输出 清除所有输出结果
-  Outline 查看所有代码块大纲
-  将图片复制至剪切板
-  图片在Syslab中直接单独查看
-  直接将图片进行导出



4.2 代码调试

Syslab提供代码调试器，支持对代码文件的单步调试、断点调试、添加监视、查看调用堆栈等，并提供交互式调试控制台。



操作步骤:

- 在脚本打上断点
- 点击菜单栏中“启动调试”或快捷键F5进入DeBug模式

说明:

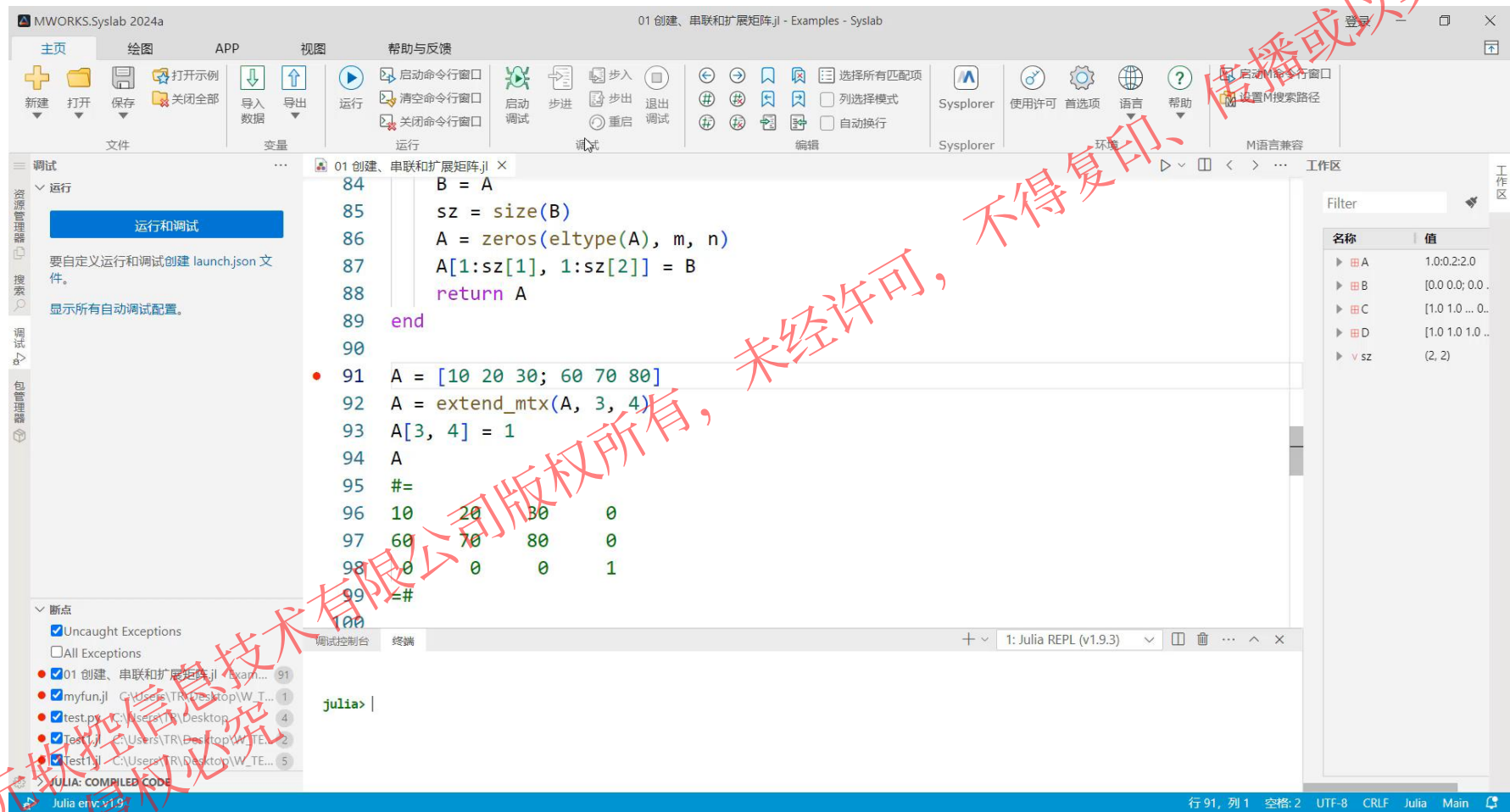
- 继续(F5): 启动调试或者继续运行调试。
- 步进(F10): 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止。
- 步入(F11): 单步执行遇到子函数就进入并且继续单步执行。
- 步出(Shift+F11): 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数。
- 重启(Ctrl+Shift+F5): 重新启动调试。
- 退出调试(Shift+F5): 停止调试。

示例见: Syslab内置 主页/打开示例/EXAMPLES/02语言基础知识/02矩阵和数组/01创建、串联和扩展矩阵



4.2 代码调试

步进(F10): 单步执行遇到子函数时不会进入子函数内, 而是将子函数整个执行完再停止。

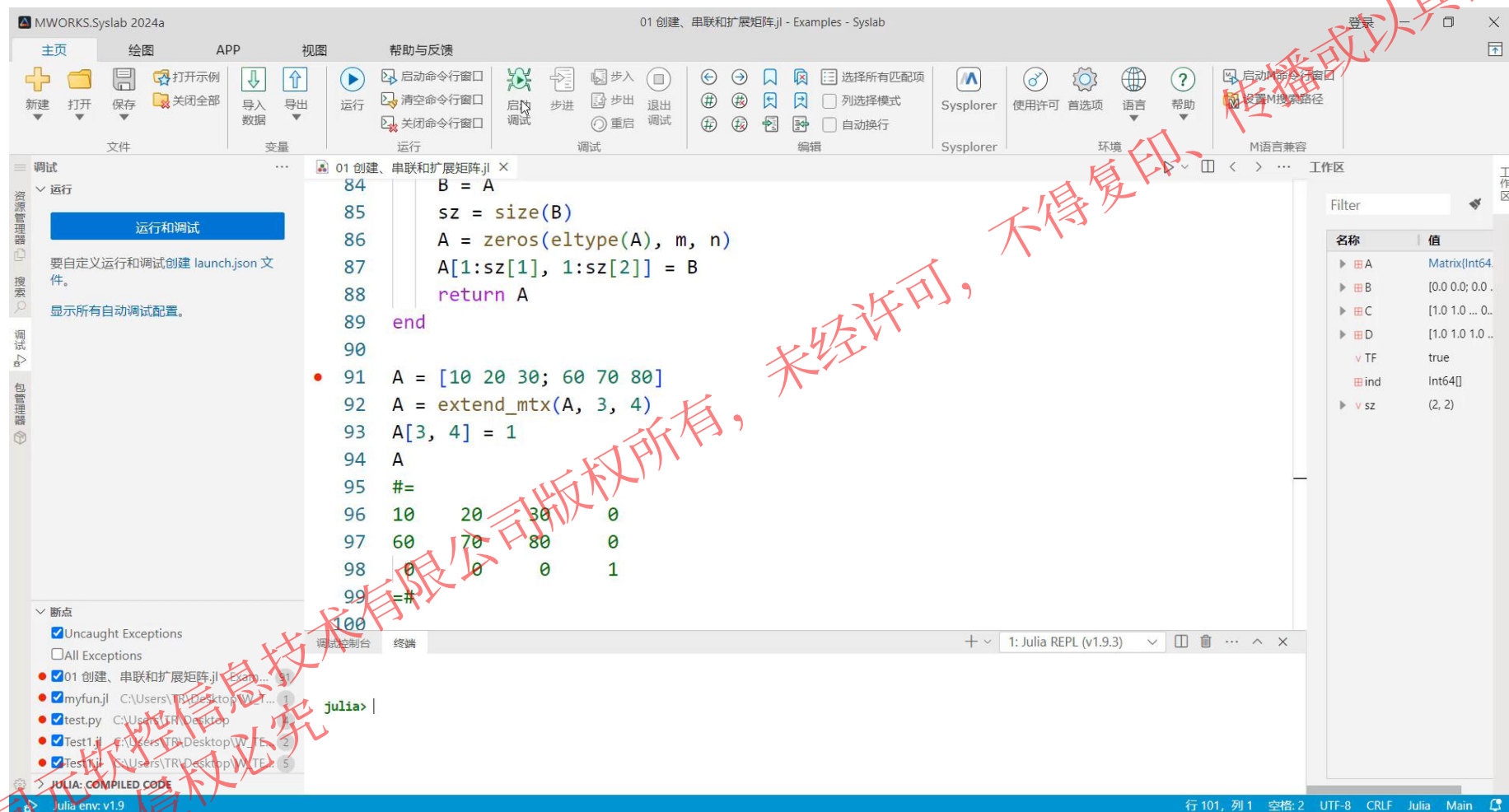


步进演示.mp4



4.2 代码调试

步入(F11): 单步执行遇到子函数就进入并且继续单步执行。



步入演示.mp4

4.2 代码调试

说明:

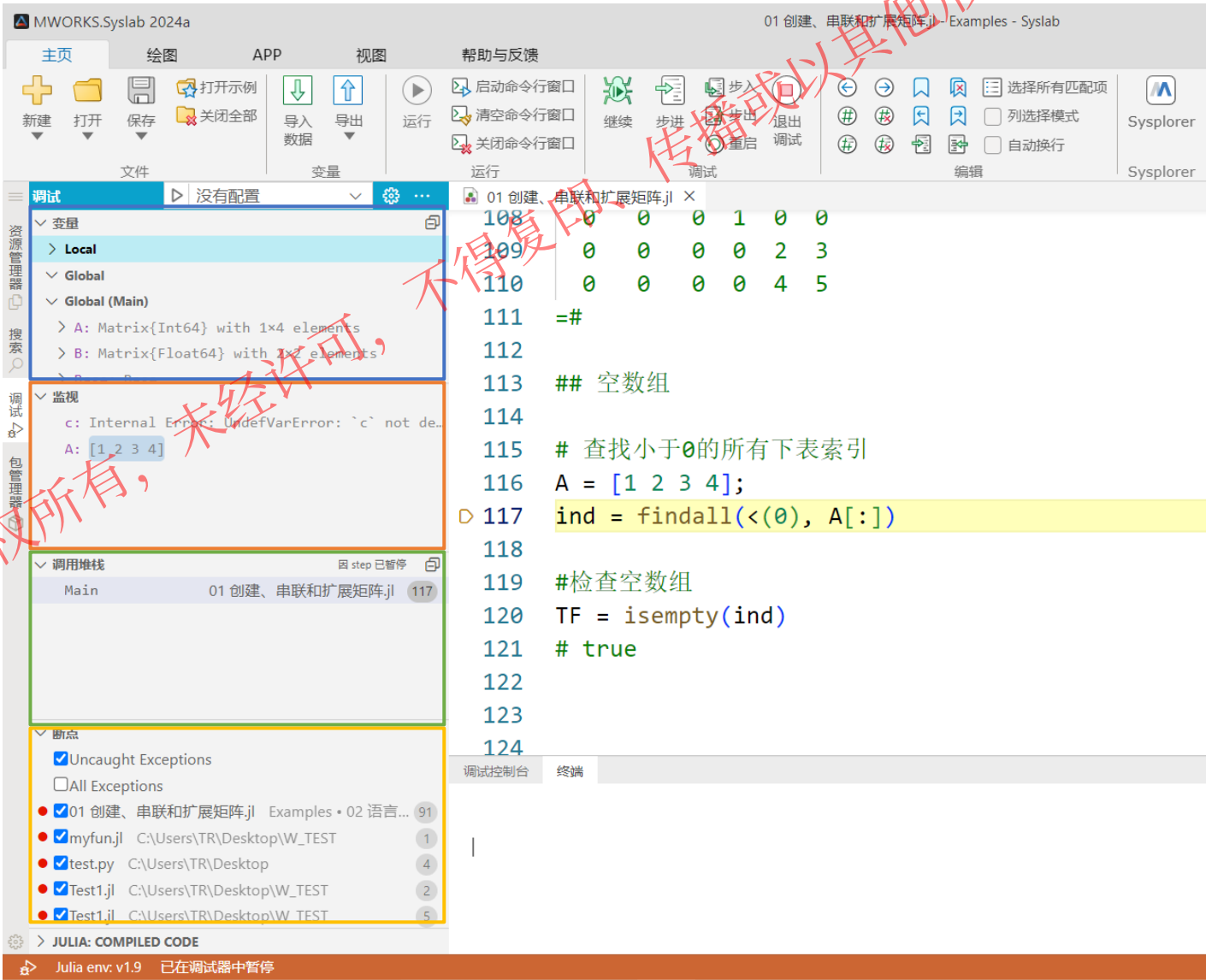
变量:

- Local: 函数局部变量的值
- Global: 函数全局变量的值, 变量前具有 global 前缀
- Global(Main): 主程序全局变量的值

监视: 输入变量名, 监视变量的变化

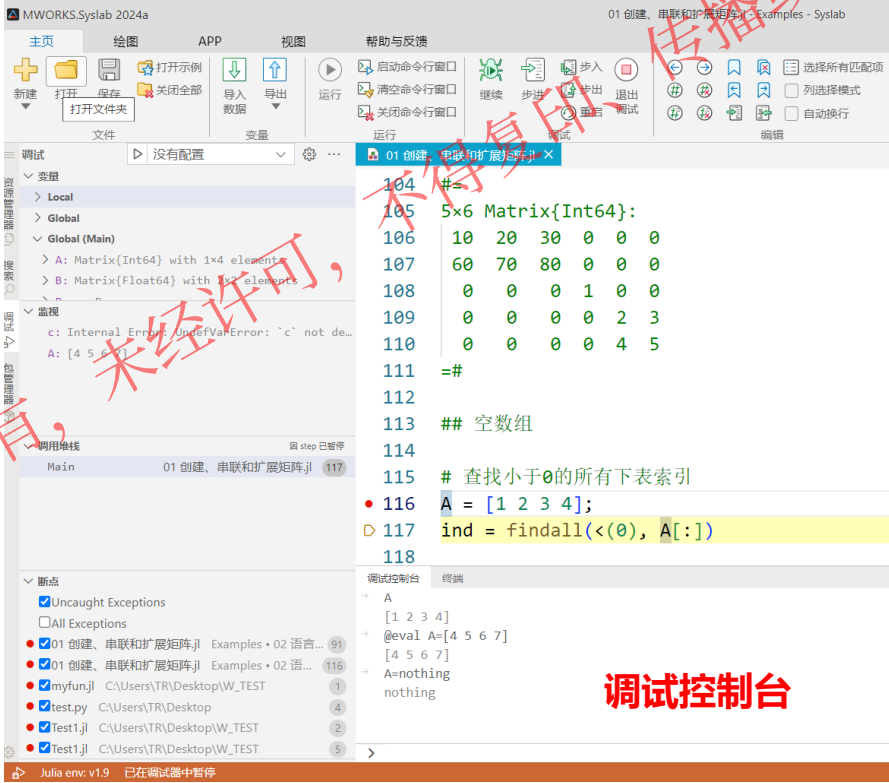
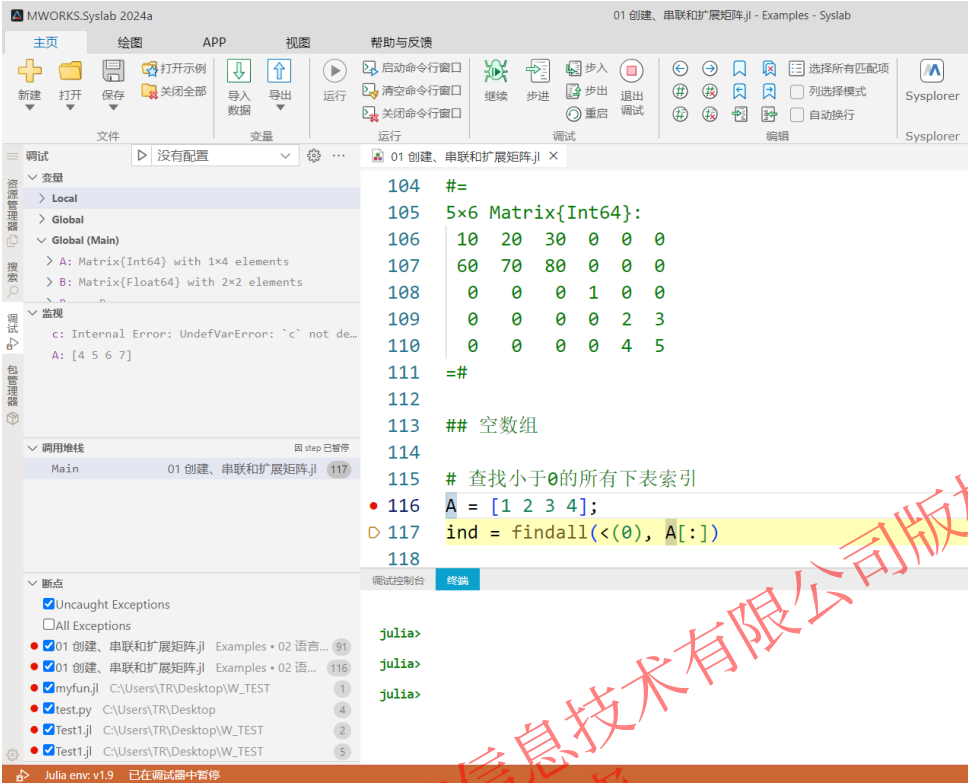
调用堆栈: 用于了解函数间的调用关系和逻辑

断点: 模型中所有断点的信息



4.2 代码调试

在调试过程中，通过调试控制台能够对变量进行增/删/改/查



调试控制台

- 查看变量值：输入变量名；
- 修改全局变量值：@eval(变量名=变量值)
- 修改局部变量值：@eval \$(变量名=变量值)
- 释放变量：变量名=nothing

目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

3. 工具箱使用

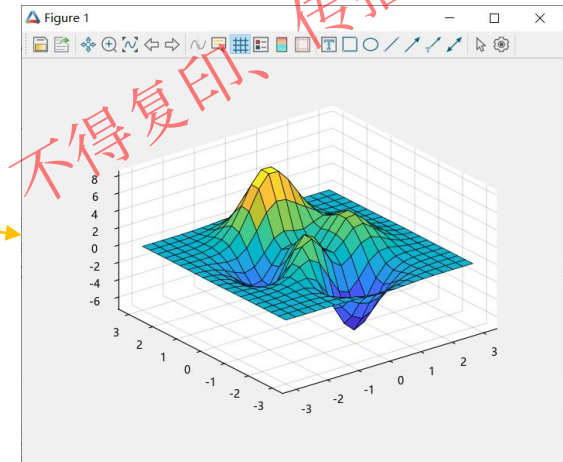
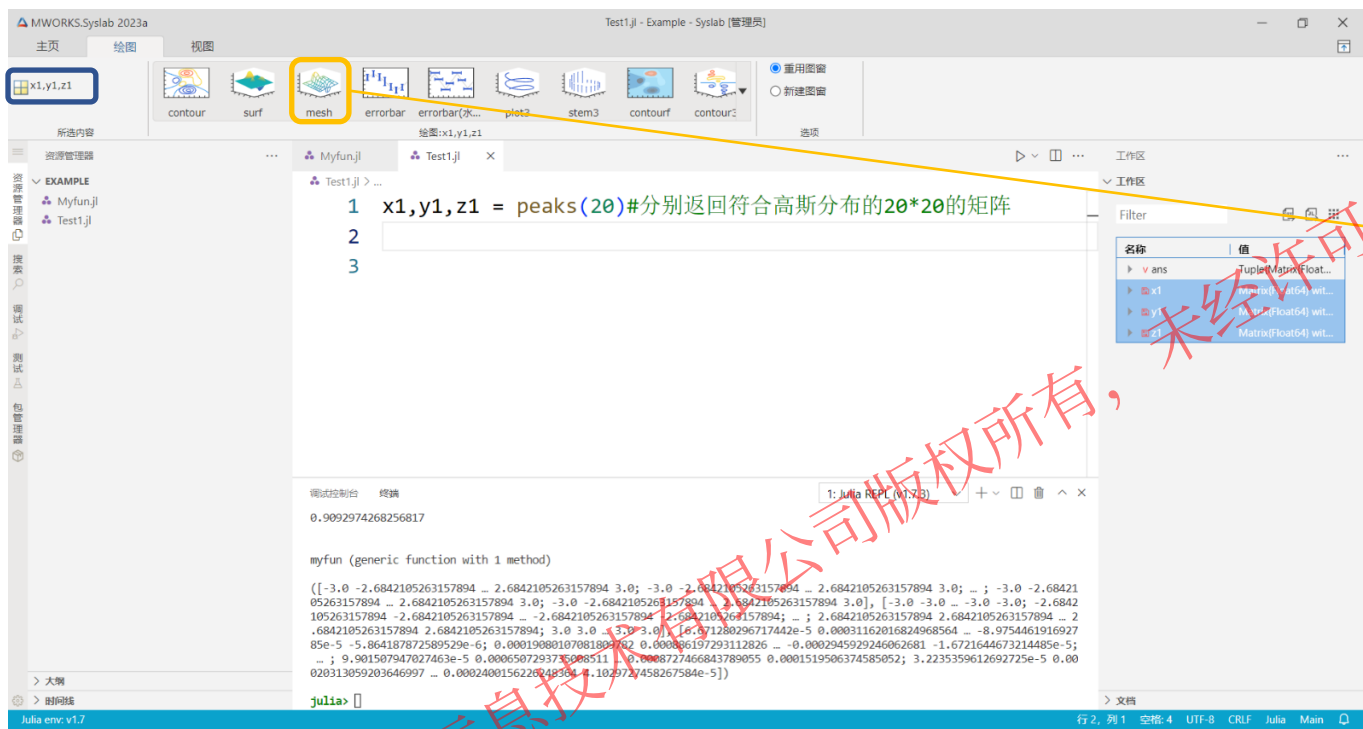
4. Julia脚本构建与调试

5. 结果后处理

6. 与其他语言的差异

5.1 结果后处理-数据可视化(图形化操作)

Syslab绘图页中具备多种类型图形绘制功能，可直接选择变量进行绘制



- 步骤:
1. 工作空间选择绘图的变量
 2. 在“绘图”菜单栏选择图的类型

`x1,y1,z1 = peaks(20)#分别返回符合高斯分布的20*20的矩阵`

5.1 结果后处理-数据可视化(图形化操作)

可在图形窗口直接配置属性，如增加标题、坐标轴标签等。



步骤:

- 1. 添加标题
- 2. 添加X、Y、Z轴标签

5.1 结果后处理-数据可视化(脚本)

二维图分类: 线图、对数图、函数图、直方图、散点图、饼图和热图、条形图、针状图、极坐标图和等高线图。

图形类型	函数	说明
线图	plot	二维线图
	stairs	阶梯图
	errorbar	含误差条的线图
	area	填充区二维绘图
对数图	loglog	双对数刻度图
	semilogx	半对数图(x轴有对数刻度)
	semilogy	半对数图(y轴有对数刻度)
函数图	fplot	绘制表达式或函数
	fimplicit	绘制隐函数
直方图	histogram	直方图
	boxchart	创建箱线图

图形类型	函数	说明
散点图	scatter	散点图
	spy	可视化矩阵的稀疏模式
	plotmatrix	散点图矩阵
饼图和热图	pie	饼图
	heatmap	创建热图
	sortx	对热图行中的元素进行排序
	sorty	对热图列中的元素进行排序
	wordcloud	使用文本数据创建文字云图
条形图	bar	条形图
	barh	水平条形图
	pareto	帕累托图
针状图	stem	针状图

图形类型	函数	说明
极坐标图	polarplot	在极坐标中绘制线条
	polarscatter	极坐标中的散点图
	polarhistogram	极坐标中的直方图
	ezpolar	易用的极坐标绘图函数
等高线图	contour	矩阵的等高线图
	contourf	填充的二维等高线图
	fcontour	绘制等高线



5.1 结果后处理-数据可视化(脚本)

可以直接使用plot函数直接绘制二维线图，一般形式为plot(x1,y1,"s1", x2,y2,"s2"),s1、s2为曲线的特征

线型	说明	表示的线条
"-"	实线	————
"--"	虚线	-----
":"	点线	
"-."	点划线	- . - . - .

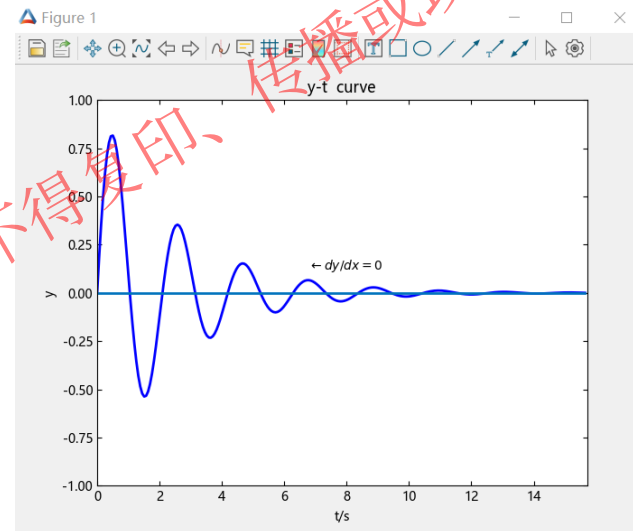
颜色名称	短名称	颜色
red	r	
green	g	
blue	b	
cyan	c	
magenta	m	
yellow	y	
black	k	
white	w	

标记	符号	说明
"."	●	空心小圆
"o"	○	空心大圆
"v"	▼	下三角
"^"	▲	上三角
"<"	◀	左三角
">"	▶	右三角
"*"	★	星号
"x"	×	叉号
"s"	■	正方形
"d"	◆	菱形

5.1 结果后处理-数据可视化(脚本)

绘制衰减图线 $y=e^{-t/2.5}\sin 3t$,其中 t 的取值范围为 $[0,5\pi]$

```
#定义自变量t的取值范围
t = 0:pi/50:5*pi
#计算对应于自变量数组的y数组, 使用广播
y = exp.(-t / 2.5) .* sin.(3t)
#绘制特征为实线、蓝色、线宽为2的曲线和0参考线
plot(t, y, "-b", linewidth = 2,t,zeros(size(t)))
#标注输出图线的最大值最小值,x轴为[0,5*pi], y轴为[-1,1]
axis([0, 5 * pi, -1, 1])
#定义xy坐标轴的名称
xlabel("t/s")
ylabel("y")
#定义图形名称
title("y-t curve")
#在(6.77,0.12)位置处插入"←dy/dx=0"
text(6.77, 0.12, raw"$\leftarrow dy/dx=0$")
#鼠标选择的位置插入文本"myplot"
gtext("myplot")
```



title —— 给图形加标题
xlabel —— 给x轴加标注
ylabel —— 给y轴加标注
text —— 在图形指定位置加标注
gtext —— 将标注加到鼠标点击位置
grid ("on/off") —— 打开、关闭坐标网格线
legend —— 添加图例
axis —— 控制坐标轴的刻度

5.1 结果后处理-数据可视化(脚本)

三维图分类: 线图、函数图、散点图、条形图、针状图、等高线图、曲面图和网格图等。

类型	函数	说明
线图	plot3	三维点或线图
函数图	fplot3	三维参数化曲线绘图函数
散点图	scatter3	三维散点图
条形图	bar3	绘制三维条形图
	bar3h	绘制水平三维条形图
针状图	stem3	三维针状图
等高线图	contour3	三维等高线图
曲面图和网格图	surf	曲面图
	mesh	网格曲面图
	peaks	包含两个变量的示例函数
	fsurf	绘制三维曲面
	fmesh	绘制三维网格图

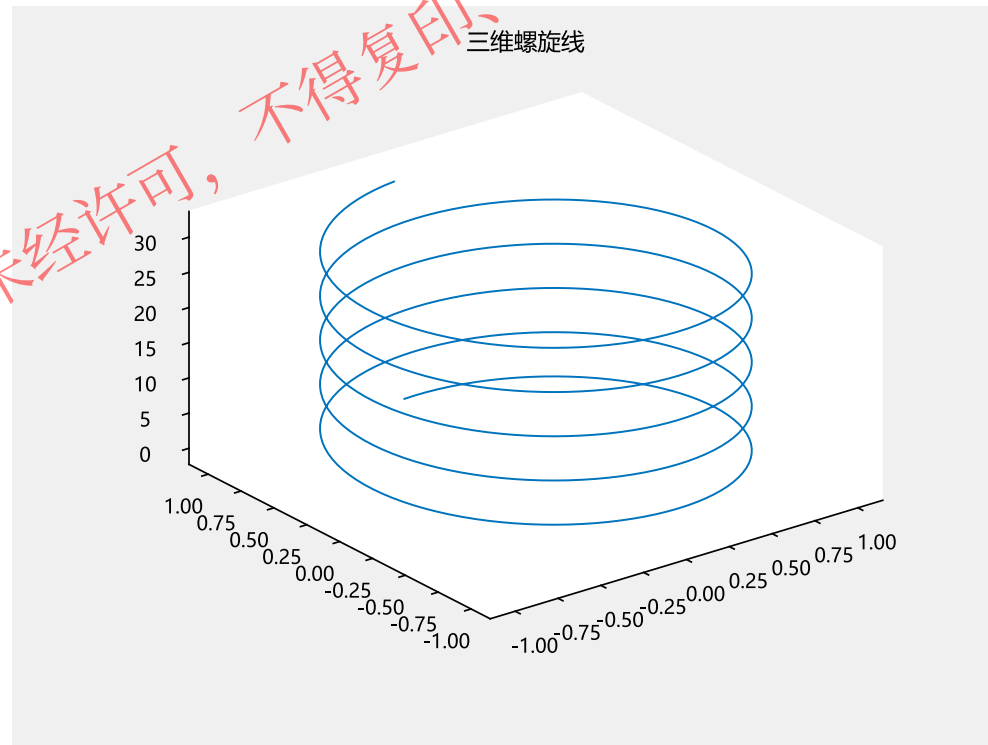


5.1 结果后处理-数据可视化(脚本)

可以直接使用plot3函数直接绘制三维线图，一般形式为plot3(x1,y1,z1,"s"), s为曲线的特征

绘制三维螺旋线

```
#定义自变量t的取值范围  
t = 0:pi/50:10*pi;  
#定义xy坐标与t的关系  
x = sin.(t);  
y = cos.(t);  
plot3(x, y, t);  
title("三维螺旋线")
```

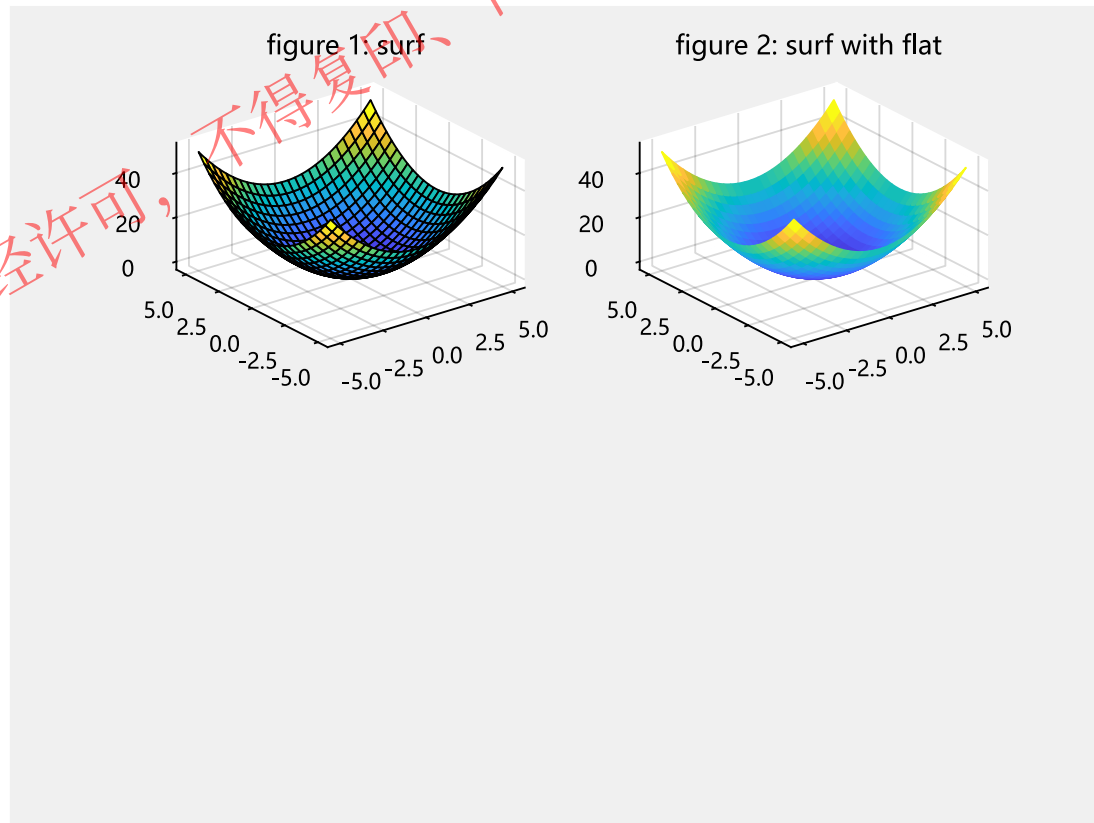


5.1 结果后处理-数据可视化(脚本)

可以使用subplot将图形区进行分区，可在各个分区中分别进行绘图

在一个图形区分别绘制普通曲面图，带平面阴影的曲面图，无网格边界的曲面图和部分被遮挡的曲面图

```
x = -5:0.2:5;  
y = x;#定义向量x和y  
X, Y = meshgrid(x, y);#生成坐标  
Z = X.^ 2 + Y.^ 2;#表达式点运算  
#图1: 普通绘制  
#subplot将多个图画到同一平面  
subplot(2, 2, 1);  
s = surf(X, Y, Z);#绘制三维图形  
title("figure 1: surf");  
#图2: 平面阴影  
subplot(2, 2, 2);  
s = surf(X, Y, Z);  
#set_edgcolor()用来修改网格属性, flat修饰网格  
s.set_edgcolor("flat")  
title("figure 2: surf with flat");
```



5.1 结果后处理-数据可视化(脚本)

可以使用subplot将图形区进行分区，可在各个分区中分别进行绘图

在一个图形区分别绘制普通曲面图，带平面阴影的曲面图，无网格边界的曲面图和部分被遮挡的曲面图

#图3: 无网格边界

```
subplot(2, 2, 3);
```

```
s = surf(X, Y, Z);
```

#none表示无网格边界

```
s.set_edgecolor("none")
```

```
title("figure 3: surf with none");
```

#图4: 遮挡绘制

```
subplot(2, 2, 4);
```

```
x1 = X[1, :];
```

```
y1 = Y[:, 1];
```

```
i = find((y1 .> 3) .& (y1 .< 4));
```

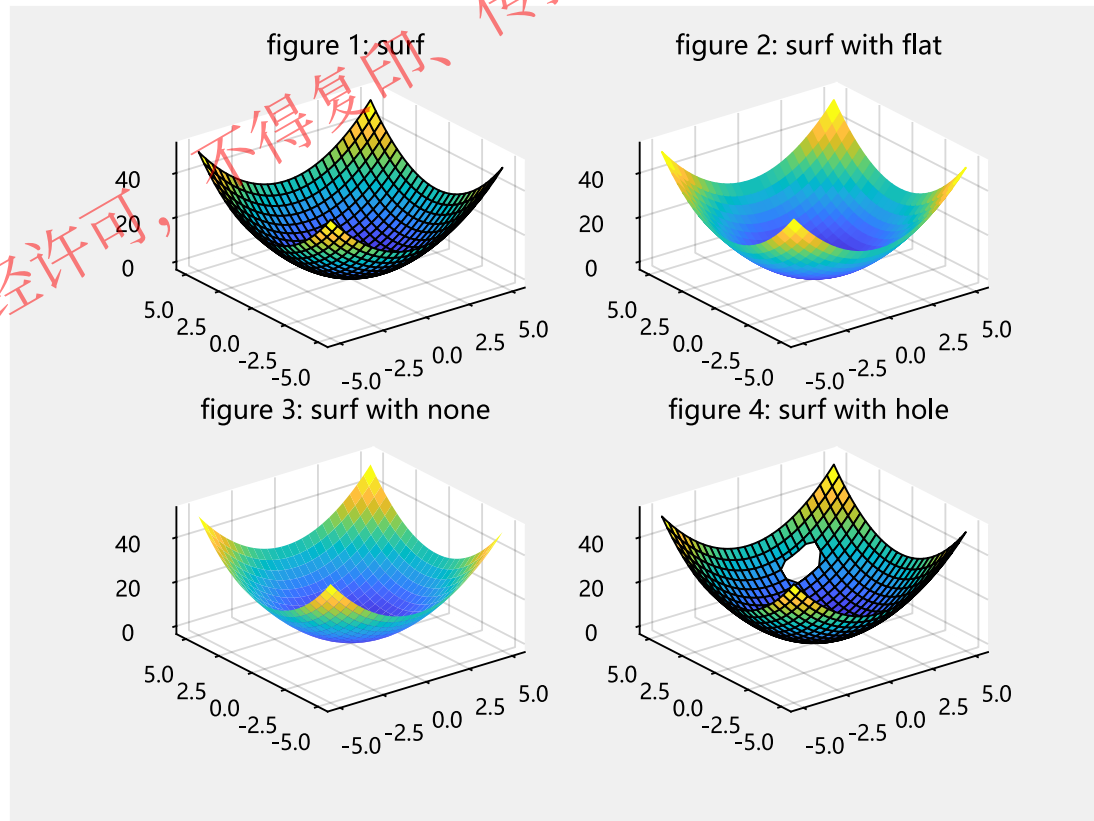
```
j = find((x1 .> 1) .& (x1 .< 3));
```

#赋空值

```
Z[i, j] .= NaN;
```

```
s = surf(X, Y, Z);
```

```
title("figure 4: surf with hole")
```

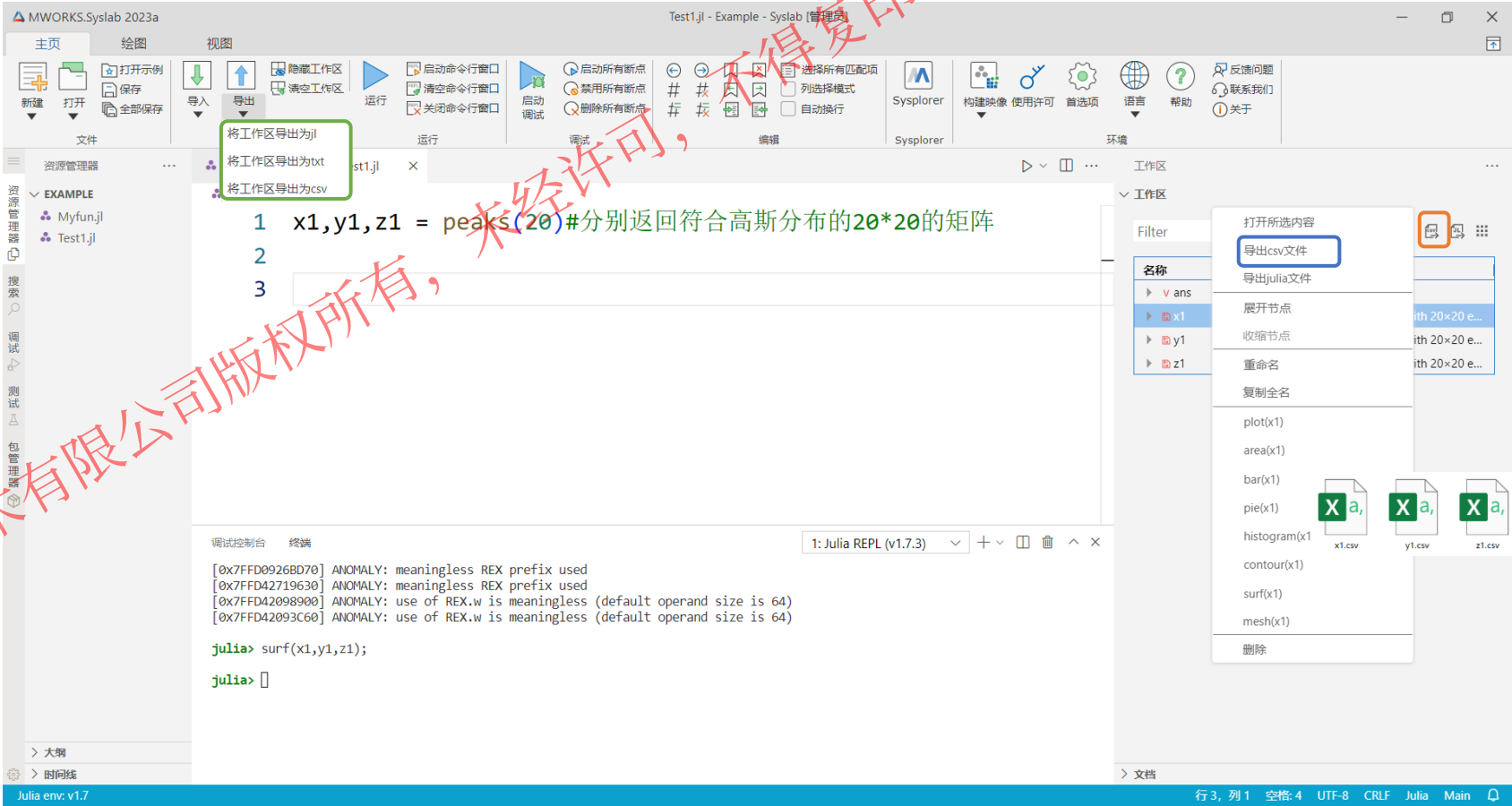


5.2 结果导出

数据导出：所有计算结果和部分变量结果可直接导出为csv格式

操作步骤：

- 方式1：导出单个结果
- 方式2：导出所有结果

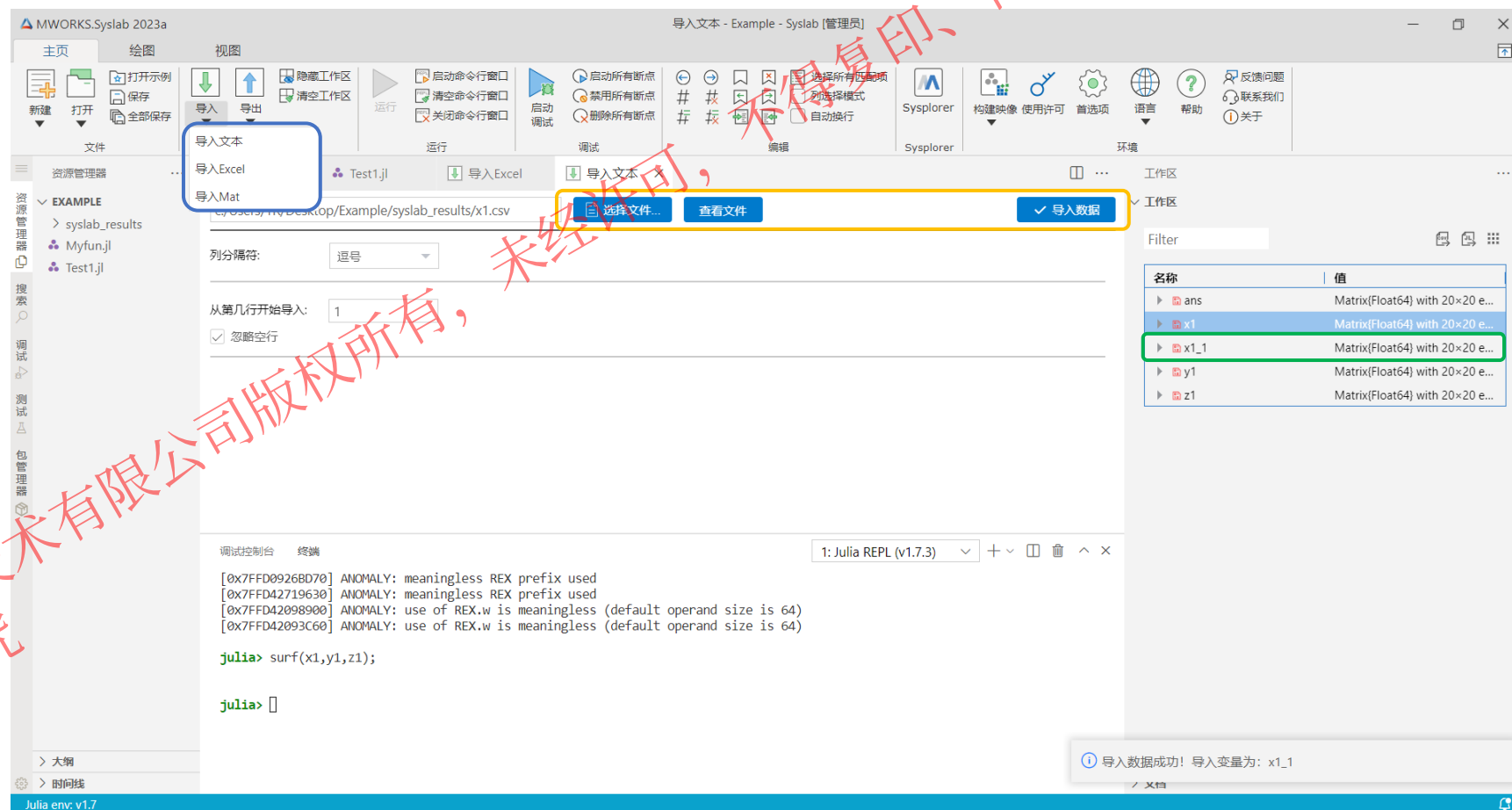


5.2 结果导入

数据导入：可将数据文件直接导入至工作空间进行使用

操作步骤：

- 点击“导入”，选择文件格式：
 - 文本：txt、csv
 - Excel：xls
 - Mat
- 选择文件并导入数据
- 工作空间查看结果，并可对结果重命名



目录

1. MWORKS.Syslab软件介绍

2. Syslab界面简介

3. 工具箱使用

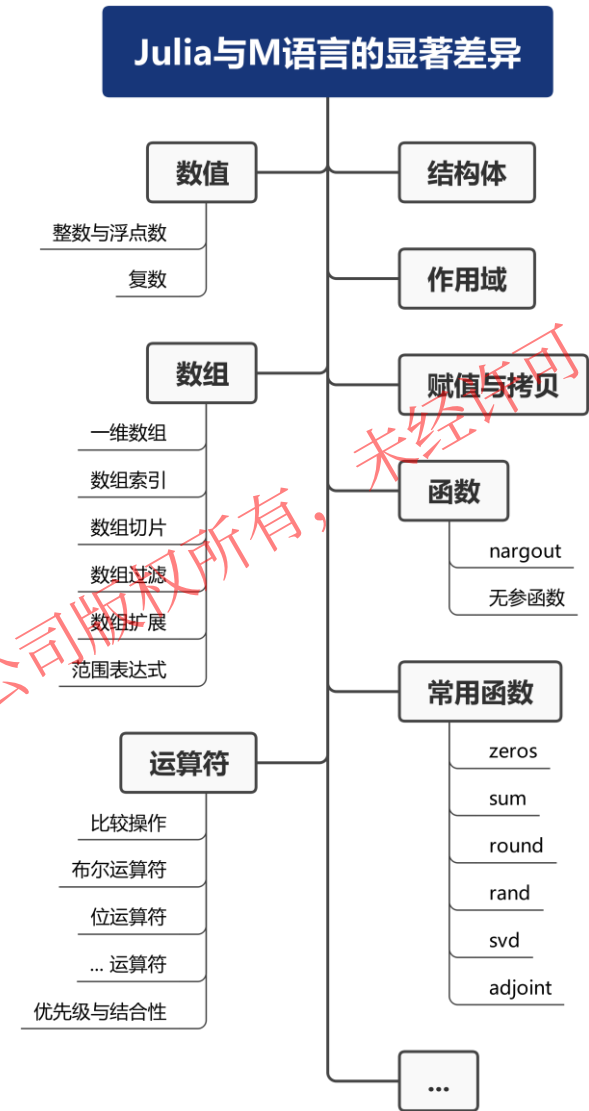
4. Julia脚本构建与调试

5. 结果后处理

6. 与其他语言的差异

6.1.1 与M语言的差异-概览

Julia 与 M 语言存在一些相似之处，但 Julia 不是 M 语言的克隆，两者存在显著差异。



6.1.2 语法差异 – 数值

- 在Julia中，没有小数点的数字字面量会**创建整数**而不是浮点数。
- 在Julia中，复数虚部为 **im**，而MATLAB中为 **i** 或 **j**。

Julia

```
a=-1
typeof(a) # Int64

# 报错
2^a
#=
ERROR: DomainError with -1:
Cannot raise an integer x to a negative
power -1.
=#

# ok
2^float(a)

b = 1 + im
```

MATLAB

```
a=-1
class(a) % 'double'

% ok
2^a

% 报错
a=int64(-1)
2^a
% Error using .^
% Integers can only be raised to positive
integral powers.

b = 1 + i
b = 1 + j
```

6.1.2 语法差异 – 数组

- **一维数组**: Julia 拥有真正的一维数组，列向量的大小为 N，而不是 Nx1。

Julia

```
a = [1 2 3]
# 1×3 矩阵

b = [1,2,3]
# 向量

c = [1;2;3]
# 向量

d = [1;2;3;;]
# 3×1 矩阵
```

MATLAB

```
a = [1 2 3]
% 1×3 矩阵

b = [1,2,3]
% 1×3 矩阵

c = [1;2;3]
% 3×1 矩阵
```



6.1.2 语法差异 – 数组

- 一维数组：Julia 拥有真正的一维数组，列向量的大小为 N，而不是 Nx1。
- **数组索引**：Julia 数组使用方括号 **A[i,j]** 进行索引，MATLAB使用 A(i,j) 进行索引。

Julia

```
a=Float64[1 2 3; 4 5 6]
#=
2×3 Matrix{Float64}:
 1.0  2.0  3.0
 4.0  5.0  6.0
=#

a[2,2]
# 5.0
```

MATLAB

```
a=[1 2 3; 4 5 6]
% a =
%      1      2      3
%      4      5      6

a(2,2)
% 5
```



6.1.2 语法差异 – 数组

- 一维数组：Julia 拥有真正的一维数组，列向量的大小为 N ，而不是 $N \times 1$ 。
- 数组索引：Julia 数组使用方括号 $A[i,j]$ 进行索引，MATLAB使用 $A(i,j)$ 进行索引。
- **数组切片**：Julia 对行向量($1 \times N$) 提取部分元素，可返回向量。

Julia

```
x = [1 2 3 4 5 6]
# 1×6 Matrix{Int64}

# 提取部分元素，返回向量
px = x[2:end]
# 5-element Vector{Int64}

# 该用法与MATLAB的x(2:end)或x(:,2:end)等价
px = x[:, 2:end]
# 1×5 Matrix{Int64}
```

MATLAB

```
x = [1 2 3 4 5 6]
% 1x6 矩阵

px = x(2:end)
% 1x5 矩阵

px = x(:,2:end)
% 1x5 矩阵
```

6.1.2 语法差异 – 数组

- 数组过滤**：在 MATLAB 中，删除不需要的值的惯用方法是使用逻辑索引，如表达式 `x(x>3)` 或语句 `x(x>3) = []` 来 in-place 修改 `x`。在 Julia 中，可以使用高阶函数 `filter` 和 `filter!`，或类似 `x[x .> 3]` 等价写法。

Julia

```
x = [1 3 5 7 9]

x[x .> 3]
# [5, 7, 9]

x = x[x .<= 3]
# x = [1, 3]
```

MATLAB

```
x = [1 3 5 7 9]

x(x>3)
% [5 7 9]

x(x>3)=[]
% x = [1 3]
```



6.1.2 语法差异 – 数组

- **数组扩展**: Julia **不会在赋值语句中自动增长数组**。而在 MATLAB 中 $a(4) = 3.2$ 可以创建数组 $a = [0 \ 0 \ 0 \ 3.2]$ 。Julia 使用 `push!`、`append!` 来增长 Vector。

Julia

```
# 针对向量
a = [0,0,0,3.2]
push!(a,7)
# [0.0, 0.0, 0.0, 3.2, 7.0]

# 针对矩阵
a = [0 0 0 3.2]
a = [a 7] # 等价于 a = hcat(a, 7)
# [0.0 0.0 0.0 3.2 7.0]
```

MATLAB

```
a = [0 0 0 3.2]
% 1x4 矩阵

a(5) = 7
% a = [0 0 0 3.2000 7.0000]
```

6.1.2 语法差异 – 数组

- **范围表达式**: 在 Julia 中, `a:b` 和 `a:b:c` 构造 `AbstractRange` 对象。使用 `collect(a:b)` 构造一个类似 MATLAB 中完整的向量。通常, 不需要调用 `collect`。在大多数情况下, `AbstractRange` 对象将像普通数组一样运行, 但效率更高, 因为它是懒惰求值。

Julia

```
x=1:2:10
# 1:2:9 (不展开, 懒惰求值, 效率高)

# 转换为向量
collect(x)
# [1, 3, 5, 7, 9]

# 创建100个元素的向量
y = LinRange(-5,5,100)

y1 = LinRange(-5,5,7)
# [-5.0, -3.33333, -1.66667, 0.0, 1.66667, 3.33333, 5.0]
```

MATLAB

```
x=1:2:10
% [1 3 5 7 9]

% 默认创建 1x100 矩阵
y = linspace(-5,5)

y1 = linspace(-5,5,7)
% [-5.0000 -3.3333 -1.6667 0 1.6667 3.3333 5.0000]
```

6.1.2 语法差异 – 结构体

- Julia的结构体 struct(不可变) 和 mutable struct(可变), 与Matlab的 struct, 差异非常大。
- 在Julia中, 对于 struct 不可变结构体, 创建后不可修改, 但性能高效。

Julia

```
# 不可变结构体
struct Foo
    baz::Int
    qux::Float64
end

# 创建实例
foo = Foo(23, 1.5)

# 不允许修改
foo.qux = 2.0
# ERROR: setfield!:
immutable struct of type
Foo cannot be changed
```

Julia

```
# 可变结构体
mutable struct Bar
    baz::Int
    qux::Float64
end

# 创建实例
bar = Bar(23, 1.5);

# 允许修改
bar.qux = 2.0
```

MATLAB

```
% 方式1: 定义结构体对象
foo.baz = 23
foo.qux = 2.5

% 方式2: 结构体对象
bar = struct("baz", 23, "qux", 1.5)

% 允许修改
bar.qux = 2.0
```



6.1.2 语法差异 – 运算符

- **比较操作**：在 Julia 中，如果 A 和 B 是数组，像 `A == B` 这样的逻辑比较运算符不会返回布尔值数组。相反地，请使用 `A .== B`。对于其他的像是 `<`、`>` 的布尔运算符同理。

Julia

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

# 数组判等
a == b
# true

# 数组元素判等 (向量化操作)
a .== b
# Bool[1, 1, 1, 1, 1]

# ok: 需要向量化操作
a .> 3
# Bool[0, 0, 1, 1, 1]
```

MATLAB

```
a=[1,3,5,7,9]
b=[1,3,5,7,9]

% 数组元素判等
a == b
% 1x5 logical array
% 1 1 1 1 1

% 数组判等
isequal(a, b)
% true

a > 3
% 1x5 logical array
% 0 0 1 1 1
```

6.1.2 语法差异 – 运算符

- **布尔运算符**: 在Julia中, 运算符采用 **!** (否定)、**&&** (短路与)、**||** (短路或) 进行布尔运算, 分别与 Matlab中的 **~**、**&&**、**||** 等价。注意, Matlab中的 **&** 与 **|** 表示无短路功能的逻辑运算, 不是位运算符。

Julia

```
A = false  
  
# 逻辑 not  
!A  
# true
```

MATLAB

```
A = false  
  
% 逻辑 not  
~A
```

3.2 语法差异 – 运算符

- **位运算符**: 在 Julia 中, 运算符 `&`、`|` 和 `⊕` (`xor`) 进行按位操作, 分别与 Matlab 中的 `bitand`、`bitor`、`bitxor` 等价。

Julia

```
# 按位取反
~4 # -5

# 按位与
4 & 10 # 0
4 & 12 # 4

# 按位或
4 | 10 # 14
4 | 1  # 5

# 按位异或
xor(4, 7) # 3
```

MATLAB

```
% 按位补码
bitcmp(int8(4)) % int8(-5)

% 按位与
bitand(int8(4), int8(10)) % int8(0)
bitand(int8(4), int8(12)) % int8(4)

% 按位或
bitor(int8(4), int8(10)) % int8(14)
bitor(int8(4), int8(1)) % int8(5)

% 按位异或
bitxor(int8(4), int8(7)) % int8(3)
```

6.1.2 语法差异 – 运算符

- **...运算符**：在Julia中，...运算符有两种用法：一是将多个参数组合成一个参数，二是将一个参数分解成多个不同参数；但在Matlab中，... 表示续行符。
- Julia没有续行符，Julia中不完整的表达式会自动延续到下一行

Julia

```
# 用法1: 将多个参数组合成一个参数
function printargs(args...)
    println(typeof(args))
    for (i, arg) in
enumerate(args)
        println("Arg # $i$  =  $arg$ ")
    end
end
printargs(10, 20, 30)
#=
Tuple{Int64, Int64, Int64}
Arg #1 = 10
Arg #2 = 20
Arg #3 = 30
=#
```

Julia

```
# 用法2: 将一个参数分解成多个不同参数
function threeargs(a, b, c)
    println("a =  $a::\text{typeof}(a)$ ")
    println("b =  $b::\text{typeof}(b)$ ")
    println("c =  $c::\text{typeof}(c)$ ")
end

x = [1, 2, 3]
threeargs(x...)
#=
a = 1::Int64
b = 2::Int64
c = 3::Int64
=#
```

Julia

```
# Julia没有续行符
y = 1 +
    2 +
    4
# 7
```

MATLAB

```
% ...表示续行符
y = 1 + ...
    2 + ...
    4
% 7
```



6.1.2 语法差异 – 运算符

- 运算符优先级：**在 Julia 中， a^b^c 被认为是 $a^{(b^c)}$ 而在 MATLAB 中它是 $(a^b)^c$ ，这是因为 Julia 的 $^$ 是右结合。建议使用 $()$ 指定优先级。

Julia

```
# Julia中, ^是右结合
4^3^2 # 等价于 4^(3^2)
# 262144

(4^3)^2
# 4096
```

MATLAB

```
% M语言中, ^是左结合
4^3^2 % 等价于 (4^3)^2
% 4096

4^(3^2)
% 262144
```



6.1.2 语法差异 – 作用域

- 在 Julia 中，每个模块有自身的全局作用域/命名空间，而在 Matlab 中只有一个全局作用域。
- 在 Julia 中，用户难以理解的问题是 **for 作用域**，参见 Syslab 帮助文档中的【[Syslab>Julia语言概览>变量作用域](#)】。

Julia REPL

```
# 在交互式环境下，如REPL
s = 0
for i = 1:10
    s += i
end
s # 55
```

Julia 文件

```
# 在非交互环境下运行，如文件
s = 0
for i = 1:10
    s += i
end
# ERROR: UndefVarError: `s`
# not defined
```

MATLAB

```
% 在文件中执行，与在命令行窗口执行，表现一致
s = 0
for i = 1:10
    s = s+i;
end
s
% 55
```



6.1.2 语法差异 – 赋值与拷贝

- Matlab 使用按值传递 (Call by value) , 防止数组被意外地篡改, 但也会导致不必要的数组拷贝。
- Julia 与 Python、Ruby等语言, 都采用共享调用技术 (Call by sharing) , 详情参见 Syslab 帮助文档中的【Syslab>Julia语言概览>赋值与拷贝】

Julia

```
# x和y指向同一块内存
x=[1,2,3]
y=x
x[1] = 10
y # [10,2,3]
```

Julia

```
# 数组拷贝
x = [1,2,3]
y = copy(x)
x[1] = 10
y # [1,2,3]
```

MATLAB

```
% 拷贝
x=[1,2,3]
y=x
x(1) = 10
y
% [1,2,3]
```



6.1.2 语法差异 – 函数

- 在 Julia 中，能返回多个值并将其赋值为元组，例如 $(a, b) = (1, 2)$ 或 $a, b = 1, 2$ 。在 Julia 中不存在 Matlab 的 `nargout`，它通常在 MATLAB 中用于根据返回值的数量执行可选工作。取而代之的是，用户可以使用可选参数和关键字参数来实现类似的功能。

Julia

```
function addme2(a,b)
    result = a+b
    absResult = abs(result)
    return result, absResult
end
```

```
addme2(1,-3)
# 返回元组 (-2, 2)
```

```
x, = addme2(1,-3)
# x = -2
```

```
x, y = addme2(1,-3)
# x = -2, y = 2
```

MATLAB

```
% 文件1
function [result,absResult] = addme2(a,b)
    result = a+b
    if nargin > 1
        absResult = abs(result);
    end
end
```

```
% 文件2 或 在命令行窗口输入
addme2(1,-3)
% 返回 -2
```

```
[x,y]=addme2(1,-3)
% x = -2, y = 2
```

6.1.2 语法差异 – 函数

- 在 Julia 中，调用无参数的函数时必须使用小括号，例如 rand()。
- 在 Matlab 中，调用无参数的函数时可以不带小括号。

Julia

```
# 函数变量
rand
# rand (generic function with 81 methods)

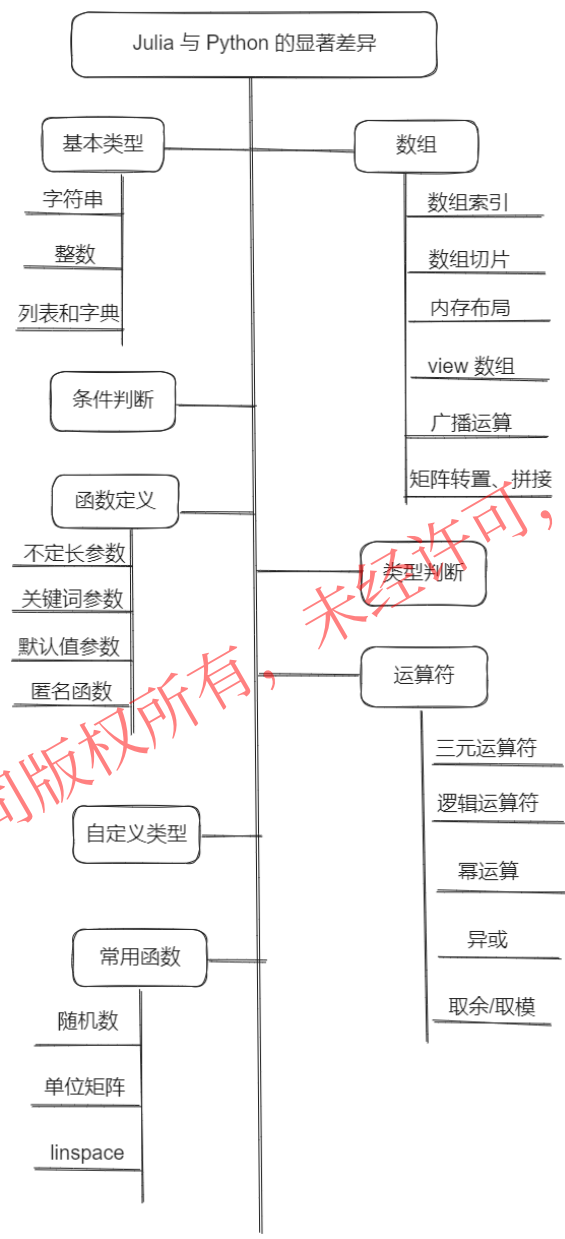
# 函数调用
rand()
# 0.3125316543482983
```

MATLAB

```
% rand 与 rand() 等价
rand % 0.8147
rand() % 0.9058
```

6.2.1 与Python的差异-概览

Python的数组实现较多，
这里采用最常用的
numpy 作为对照。



6.2.2 语法差异 – 字符串

- **字符串**: 在 Julia 中, 使用双引号 "text" 表示字符串, 单引号 'c' 表示字符
在 Python 中, 字符串可以使用单引号或者双引号来表示

Julia

```
# 字符串
S = "text"

# 字符
C = 'c'

# 报错, ''只能包含单个字符
C = 'text'
#=
ERROR: syntax: character literal contains
multiple characters
=#
```

Python

```
# 字符串
S = "text"

# 字符串
C = 'text'
```

6.2.2 语法差异 – 字符串

- **字符串拼接**: 在 Julia 中, 字符串拼接使用 `*`, 字符串重复n次使用 `^`
在 Python 中, 字符串拼接使用 `+`, 字符串重复n次使用 `*`

Julia

```
# 字符串拼接
S = "text" * "123" # "text123"

# 字符串重复 3 次
S = "ab" ^ 3 # "ababab"
```

Python

```
# 字符串拼接
S = "text" + "123" # "text123"

# 字符串重复 3 次
S = "ab" * 3 # "ababab"
```

6.2.2 语法差异 – 整数

- **整数**: 在 Julia 中, Int 类型是 32位/64 位有符号整数, 整数运算可能溢出, 需要根据场景改用大整数 BigInt 或者浮点数类型; 在 Python 中, int 类型是无限精度类型, 但 numpy 中的 int 类型默认为 64/32 位 (linux上默认为 64 位, windows上默认为 32 位)

Julia

```
# 默认为 32 位/64 位整数类型
typeof(2) # Int32/Int64

# 整数运算溢出
2^64 # 0

# 如果需要可使用 BigInt
big(2)^64 # 18446744073709551616
```

Python

```
import numpy as np
# 内置的整数类型不会溢出
2**64 # 18446744073709551616

# numpy 的整数类型为 32 位或者 64 位
# windows 上为 dtype('int32')
# linux 上位 dtype('int64')
x = np.array(2)
x.dtype

# 整数运算可能溢出
x**64 # 0
```


6.2.2 语法差异 – 列表和字典

- **存储不同类型的元素**: 在 Python 中, 列表和字典内可以存储不同类型的元素, 在 Julia 中, 存储不同类型元素的容器可以使用 `Vector{Any}` 或者 `Dict{String,Any}`

Julia

```
# 使用 Any 作为容器的元素类型会降低 Julia 代码的运行速度
# 在使用前请确认容器是否真的需要存储不同类型的元素

# 创建可存储任意元素的列表
list = Any[1, "abc", 2.1]
# 往列表内添加元素
push!(list, "text")

# 创建可存储任意元素的字典
dict = Dict{String, Any}(
    "a" => 1, "b" => "abc")
```

Python

```
# 创建可存储任意元素的列表
list = [1, "abc", 2.1]

# 往列表内添加元素
list.append("text")

# 创建可存储任意元素的字典
dict = {"a": 1, "b": "abc"}
```

6.2.2 语法差异 – nothing 和 none

- **常见的空值表示**: 在 Julia 中, 常使用 **nothing** 来表示空值
在 Python 中, 常使用 **None** 来表示空值

Julia

```
x = nothing

# 使用 !isnothing(x) 来判断值是不是 nothing
if !isnothing(x)
    println("abc")
end
```

Python

```
x = None

# 使用 x is not None 来判断值是不是 None
if x is not None:
    print("abc")
```

6.2.2 语法差异 – 数组

- **数组字面量**: 在 Julia 中, 可以使用 `[1,2,3]` 来创建向量;
在 Python 中, 可以使用 `np.asarray([1,2,3])` 来从列表中创建数组

Julia

```
# 创建向量 [1,2,3]
x = [1, 2, 3]

# 创建 2x3 的矩阵
x = [1 2 3;
      4 5 6]
```

Python

```
import numpy as np

# 创建向量 [1,2,3]
np.asarray([1,2,3])

# 创建 2x3 的矩阵
np.asarray([[1,2,3],[4,5,6]])
```

6.2.2 语法差异 – 数组

- **数组索引**: 在 Julia 中, 数组索引从 1 开始; 在 Python 中, 数组索引从 0 开始
- **数组切片**: 在 Julia 中, 数组切片包含最后一个索引; 在 Python 中, 数组切片不包含最后一个索引
- **负数索引**: 在 Julia 中, 不支持负数索引, 可以使用 end 来索引数组的最后一位

Julia

```
x = [1, 2, 3, 4, 5]

# 取第 1 个元素
x[1] # 1

# 取最后一个 (第 5 个) 元素
x[end] # 5

# 取第 2 个到第 4 个元素
x[2:4] # [2, 3, 4]

# 取第 2 个到最后一个元素
x[2:end] # [2, 3, 4, 5]
```

Python

```
import numpy as np
x = np.asarray([1,2,3,4,5])

# 取第 1 个元素
x[0] # 1

# 取最后一个 (第 5 个) 元素
x[-1] # 5

# 取第 2 个到第 4 个元素
x[1:4] # [2, 3, 4]

# 取第 2 个到最后一个元素
x[1:] # [2, 3, 4, 5]
```

6.2.2 语法差异 – 数组

- **切片步长**: Julia 可以使用 `x[start:step:stop]` 来表示间隔 `step` 步长的切片, Python 使用 `x[start:(stop+1):step]` 来表示带步长的切片

Julia

```
x = [1, 2, 3, 4, 5, 6]

# 取偶数位的元素
x[2:2:end] # [2, 4, 6]

# 数组翻转, 也可使用 reverse(x)
x[end:-1:1] # [6, 5, 4, 3, 2, 1]
```

Python

```
import numpy as np
x = np.asarray([1,2,3,4,5,6])

# 取偶数位的元素
x[1::2] # [2, 4, 6]

# 数组翻转, 也可使用 np.flip(x)
x[::-1] # [6, 5, 4, 3, 2, 1]
```

6.2.2 语法差异 – 数组

- **数组内存布局**: Julia 使用**列主序**来存储数据和遍历数组，和Fortran、Matlab相同，Python 默认使用**行主序**来存储数据和遍历数组，也兼容列主序的内存布局

Julia

```
A = [1 2 3; 4 5 6]
```

`vec(A)` # [1,4,2,5,3,6]

1	2	3
4	5	6

列主序

1
4
2
5
3
6

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])
```

`A.flatten()` # [1,2,3,4,5,6]

1	2	3
4	5	6

行主序

1
2
3
4
5
6

6.2.2 语法差异 – 数组

- **线性索引**: Julia 矩阵的索引 $A[i]$ 表示线性索引, Python 对矩阵取线性索引使用 $A.flat[i]$
- **按行切片**: Julia 使用 $A[i,:]$ 取矩阵的第 i 行, Python 可使用 $A[i]$ 或者 $A[i,:]$ 来取矩阵的第 $i+1$ 行

Julia

```
A = [1 2 3; 4 5 6]

# 线性索引取第 3 个元素
# 按列主序方向遍历
A[3] # 2

# 取矩阵的第 2 行
A[2,:] # [4, 5, 6]
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 线性索引取第 3 个元素
# 按行主序方向遍历
A.flat[2] # 3

# 取矩阵的第二行
A[1] # [4,5,6]
A[1,:] # [4,5,6]
```


6.2.2 语法差异 – 数组

- **view数组**: Julia 的数组切片默认返回一个新的数组, 可使用 @view 宏来返回数组的视图, 即切片和原数组共享一份数据; Python 的 numpy 数组在切片时默认返回原数组的视图

Julia

```
A = [1 2 3; 4 5 6]

# 创建新的数组, 修改 B 不会影响 A
B = A[1, :]
B[1] = 100

# 创建视图, 修改 C, A 也会被修改
C = @view A[1, :]
C[1] = 100

#=
A = [100 2 3;
     4 5 6]
=#
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 创建新的数组, 修改 B 不会影响 A
B = np.copy(A[0, :])
B[0] = 100

# 创建视图, 修改 C, A 也会被修改
C = A[0, :]
C[0] = 100

"""
A = [[100, 2, 3 ],
     [4,   5,  6]]
"""
```

6.2.2 语法差异 – 数组

- **广播运算**：Julia 的数组之间广播运算需要使用 `.+, .-, .*, ./` 等带点运算符；
Python 的数组之间广播运算无需加点；
一维向量和二维矩阵进行广播运算时，Julia 和 Python 的向量扩展方向不同。

Julia

```
A = [1 2 3; 4 5 6]

# 每个元素都增加 1
A .+ 1 # [2 3 4; 5 6 7]

# 第 1,2,3 列分别乘以 1,2,3
A .* [1 2 3] # [1 4 9; 4 10 18]

# 和 1 维向量广播时
# 1 维向量扩展为 nx1 的矩阵
# 第 1,2 行分别减去 1,2
A .- [1, 2] # [0 1 2; 2 3 4]
```

Python

```
import numpy as np
A = np.asarray([[1,2,3],[4,5,6]])

# 每个元素都增加 1
A + 1 # # [[2, 3, 4], [5, 6, 7]]

# 第 1,2,3 列分别乘以 1,2,3
A * np.asarray([[1, 2, 3]])
# [[1, 4, 9], [4, 10, 18]]

# 和 1 维向量广播时
# 1 维向量扩展为 1xn 的矩阵
# 第 1,2,3 列分别减去 1,2,3
A - np.asarray([1, 2, 3])
# [[0, 0, 0], [3, 3, 3]]
```



6.2.2 语法差异 – 数组

- **矩阵乘法**：Julia 使用 `A * B` 表示矩阵乘法，Python 使用 `A @ B` 表示矩阵乘法；

Julia

```
A = [1 2; 3 4]
B = [0 1; 1 0]

# 矩阵乘法
A * B
# [2 1;
#  4 3]
```

Python

```
import numpy as np
A = np.asarray([[1,2],[3,4]])
B = np.asarray([[0,1],[1,0]])

# 矩阵乘法
A @ B
#[[2, 1],
# [4, 3]]
```



6.2.2 语法差异 – 数组

- 矩阵转置：** Julia 使用 A' 表示矩阵的共轭转置，`transpose(A)` 表示矩阵的转置，Python 使用 `A.T` 表示矩阵的转置，`A.T.conjugate()` 表示矩阵的共轭转置

Julia

```
A = [1.0+1.0im 2.0+1.0im;
      4.0+1.0im 5.0+1.0im]

# 转置
transpose(A)
permutedims(A)
# [1.0+1.0im 4.0+1.0im;
#   2.0+1.0im 5.0+1.0im]

# 共轭转置
A'
adjoint(A)
# [1.0-1.0im 4.0-1.0im;
#   2.0-1.0im 5.0-1.0im]
```

Python

```
import numpy as np
A = np.asarray([[1.0+1.0j, 2.0+1.0j],
                [4.0+1.0j, 5.0+1.0j]])

# 转置
A.T
np.transpose(A)
# [[1.+1.j, 4.+1.j],
#   [2.+1.j, 5.+1.j]]

# 共轭转置
A.T.conjugate()
# [[1.-1.j, 4.-1.j],
#   [2.-1.j, 5.-1.j]]
```

6.2.2 语法差异 – 数组

- 数组拼接：** Julia 可以使用 `[x y]`, `[x; y]` 或者 `vcats(x,y)`, `hcat(x,y)` 来拼接矩阵，而在 Python 中需要使用 `np.vstack((x,y))` 或者 `np.hstack((x,y))`。

Julia

```
x = [1. 2. 3.;
      4. 5. 6.]

# 垂直拼接
[x; ones(1,3)]
vcat(x, ones(1,3))
# [1. 2. 3.;
#   4. 5. 6.;
#   1. 1. 1.]

# 水平拼接
[x ones(2,1)]
hcat(x, ones(2,1))
# [1. 2. 3. 1.;
#   4. 5. 6. 1.]
```

Python

```
import numpy as np
x = np.asarray([[1.,2.,3.],[4.,5.,6.]])

# 垂直拼接
np.vstack((x, np.ones((1,3))))
# [[1.,2.,3.],
#   [4.,5.,6.],
#   [1.,1.,1.]]

# 水平拼接
np.hstack((x, np.ones((2,1))))
# [[1.,2.,3.,1.],
#   [4.,5.,6.,1.]]
```

6.2.2 语法差异 – 条件判断语句

- **if 语句**: 在 Julia 中, if 关键词之后一定需要一个 Bool 类型的表达式
在 Python 中, if 关键词之后的表达式会隐式转换为 bool 类型

Julia

```
# 报错, if 关键词后不能使用非 Bool 类型变量
txt = "something"
if txt
    println("abc")
end
# ERROR: TypeError: non-boolean (String)
# used in boolean context

# 使用 !isempty(txt) 来表示字符串非空的条件
txt = "something"
if !isempty(txt)
    println("abc")
end
```

Python

```
# txt 字符串非空时, 条件成立
txt = "something"
if txt:
    print("abc")

# txt 字符串为空, 条件不成立
txt = ""
if txt:
    print("edf") # 不打印
```

6.2.2 语法差异 – 函数参数

- **不定长参数**: 在 Julia 中, 定义函数时, 在最后一个形参名后加上 ... 表示不定长参数
在 Python 中, 定义函数时, 在最后一个形参名前加上 * 表示不定长参数

Julia

```
# 最后一个形参名 args 后加 ... 表示不定长参数
function add(x, args...)
    res = 0
    for i in args
        res += i
    end
    return res + 2*x
end
println(add(1, 2, 3, 4, 5)) # 16
```

Python

```
# 最后一个形参名 args 前加 * 号表示不定长参数
def add(x, *args):
    res = 0
    for i in args:
        res += i
    return res + 2*x
print(add(1, 2, 3, 4, 5)) # 16
```


6.2.2 语法差异 – 函数参数

- **关键词参数**：在 Julia 中，定义函数时，在 ; 之后的参数为关键词参数，kwargs... 表示不定长的关键词参数。
在 Python 中，定义函数时，无需特意指定关键词参数，不定长的关键词参数使用 **kwargs 表示。

Julia

```
# 关键词参数在 ; 之后
function mul(; x, y, kwargs...)
    z = 0
    if haskey(kwargs, :z)
        z = kwargs[:z]
    end
    return x + 2 * y + z
end

mul(y = 2, x = 1, z = 1)
# 1 + 2*2 + 1 = 6
```

Python

```
# 不定长的关键词参数使用 **kwargs 表示
def mul(x, y, **kwargs):
    z = 0
    if "z" in kwargs:
        z = kwargs["z"]

    return x + 2 * y + z

mul(y = 2, x = 1, z = 1)
# 1 + 2*2 + 1 = 6
```



6.2.2 语法差异 – 函数参数

- **函数的默认值参数**：在 Julia 中，默认值参数**每次**运行时都会执行；
在 Python 中，默认值参数仅在**第一次**运行时会执行

Julia

```
function func(x=rand())  
    println(x)  
end
```

```
# 两次运行 x 的值不相同  
# 每次运行都会调用一次 rand()  
func()  
func()
```

Python

```
import numpy as np  
def func(x=np.random.rand()):  
    print(x)
```

```
# 两次运行时的 x 都相同  
# np.random.rand() 仅执行了一次  
func()  
func()
```

6.2.2 语法差异 – 匿名函数

- **匿名函数**: 在 Julia 中, 使用 $(x) \rightarrow f(x)$ 表示匿名函数
在 Python 中, 使用 `lambda x: f(x)` 表示匿名函数

Julia

```
# 匿名函数, () 内为形参
myf = (x) -> x+1
myf(1) # 2
```

Python

```
# 匿名函数, : 之前为形参
myf = lambda x: x+1
myf(1) # 2
```

6.2.2 语法差异 – 类型判断

- Julia 使用 isa 函数判断类型，Python 使用 isinstance 函数判断类型

Julia

```
function func(x)
    # 使用 isa 判断 x 是否是 Int 类型
    if x isa Int
        println("Int")
    else
        println("not Int")
    end
end

func(1) # "Int"
```

Python

```
def func(x):
    # 使用 isinstance 判断 x 是否是 int 类型
    if isinstance(x, int):
        print("int")
    else:
        print("not int")

func(1) # "int"
```

6.2.2 语法差异 – 结构体

- Julia 没有 class，但可以使用 struct / mutable struct 定义结构体，结构体定义后不能增减字段，类似 C 的 struct 定义。
- Python 使用 class 定义类，类定义后能增减属性。

Julia

```
# 定义可变结构体
mutable struct Foo
    baz::Int
    qux::Float64
end

# 创建实例
foo = Foo(23, 1.5)

# 不允许增加字段
foo.abc = 3
# ERROR: type Foo has no field abc
```

Python

```
# 定义 class
class Foo():
    baz: int
    qux: float
    def __init__(self, baz, qux):
        self.baz = baz
        self.qux = qux

# 创建实例
foo = Foo(23, 1.5)

# 新增字段
foo.abc = 3

foo.abc # 3
```

6.2.2 语法差异 – 方法分派

- Julia 使用多重分派机制，根据所有参数的运行时类型决定调用的方法实现
- Python 一般根据方法接收者的类型，或者 instance 函数来实现分派

Julia

```
struct Bar end

# 定义两个方法实现
func(bar::Bar, y::Int) = 1
func(bar::Bar, y::String) = 2

# 根据所有参数的类型来派发
func(Bar(), 1) # 1
func(Bar(), "abc") # 2
```

Python

```
class Bar:
    def func(self, x):
        if isinstance(x, int):
            return 1
        elif isinstance(x, str):
            return 2

bar = Bar()
bar.func(1) # 1
bar.func("abc") # 2
```

6.2.2 语法差异 – 三元运算符

- **三元运算符**: 在 Julia 中, 三元运算符使用 `cond ? then_val : else_val` 表示, 和 C 相同
在 Python 中, 三元运算符使用 `then_val if cond else else_val` 表示

Julia

```
# 三元运算符
true ? 1 : 0 # 1
```

Python

```
# 三元运算符
1 if True else 0 # 1
```



6.2.2 语法差异 – 运算符

- 在 Julia 中, ! 表示取非, ^ 表示幂运算, ÷ 表示整除, ∨ 表示异或运算, % 表示取余
- 在 Python 中, not 表示取非, ** 表示幂运算, // 表示整除, ^ 表示异或运算, % 表示取模

Julia

```
!false # 取非, true
2 ^ 2  # 幂运算, 4
3 ÷ 2   # 整除 (使用\div<tab>输入), 1
1 ∨ 0   # 异或 (使用\xor<tab>输入), 1
7 % -2  # 取余, 1
```

Python

```
not False # 取非, true
2 ** 2    # 幂运算, 4
3 // 2     # 整除, 1
1 ^ 0      # 异或, 1
7 % -2     # 取模, -1
```



6.2.3 常用函数差异 – rand函数

- 在 Julia 中, 使用 Random 模块相关函数生成随机数
- 在 Python 中, 使用 numpy.random 模块生成随机数

Julia

```
import TyRandom, Random
# 设置全局随机数生成器的 seed
Random.seed!(1234)

# 使用全局随机数, 生成 2x3 的均匀分布随机数
randn((2,3))

# 新建 seed 为 42 的随机数生成器
rng = TyRandom.MT19937ar(42)

#使用随机数生成器rng, 生成 2x4 的正态分布随机数
randn(rng, (2,4))

#使用随机数生成器rng, 生成 2x3 的均匀分布随机数
rand(rng, (2, 3))
```

Python

```
import numpy as np
# 设置全局随机数生成器的 seed
np.random.seed(1234)

# 使用全局随机数, 生成 2x3 的均匀分布随机数
np.random.normal(size=(2,3))

# 新建 seed 为 42 的随机数生成器
rng = np.random.default_rng(seed=42)

# 使用随机数生成器rng, 生成 2x4 的正态分布随机数
rng.normal(size=(2, 4))
# 使用随机数生成器rng, 生成 2x3 的均匀分布随机数
rng.random(size=(2, 3))
```

6.2.3 常用函数差异 – 单位矩阵

- 在 Julia 中，可使用 LinearAlgebra 模块内的 `I` 变量表示单位矩阵
- 在 Python 中，使用 `numpy.eye` 函数创建单位矩阵

Julia

```
using LinearAlgebra
A = [1. 2.; 3. 4.]
# 可使用 I 表示单位矩阵
A + I
# [2.0  2.0
#  3.0  5.0]
```

Python

```
import numpy as np
A = np.asarray([[1.,2.],[3.,4.]])
A + np.eye(2)
# [[2., 2.],
#  [3., 5.]]
```

6.2.3 常用函数差异 – linspace

- 在 Julia 中, 可使用 `range(start, stop, length)` 函数实现 numpy 的 `linspace` 功能

Julia

```
collect(range(-0.1, 0.3, length=5))  
# [-0.1, 0. , 0.1, 0.2, 0.3]
```

Python

```
import numpy as np  
np.linspace(-0.1, 0.3, num=5)  
# [-0.1, 0. , 0.1, 0.2, 0.3]
```

建立知识规范，营造协同生态

积累工业模型，发展可控平台

融入中国创新，打造先进软件

请各位专家指正！

同元软控 | 为世界提供科学计算 与系统建模仿真平台的中国选项



扫码关注
同元软控公众号



扫码关注
MWORKS公众号



扫码关注
同元招聘公众号



扫码关注
同元软控视频号



扫码前往
同元软控官网



扫码前往
MoHub官网



bilibili 搜索
同元软控



知乎搜索
同元软控



小红书搜索
同元软控



抖音搜索
同元软控

传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究